



[하만세미콘아카데미] 3주차_과제레포트

About UVM

[14조] 김준회(팀장), 이승후, 윤호승, 조현우

Contents

1. UVM Phase

2. UVM Factory

3. UVM Config

4. UVM Macro, Methods

5. UVM Component

6. UVM Communication mechanism

7. UVM Sequence

8. UVM RAL

9. Virtual Sequencer & Sequence



1. **UVM Phase**

01

1) Concept of Phasing

● What is Phase?

모든 Component가 공통적으로 따라가는 일련의 시뮬레이션 life-cycle

- 1. 우선 Component가 정의되고 instantiation 되어야 연결할 수 있고, (build-time phases)
phases)
- 2. 연결이 되어야 transaction을 test sequence에 따라 내보낼 수 있으며, (run-time phases)
phases)
- 3. transaction이 모두 완료된 시점에서야 error 여부를 최종적으로 check 하고 report를 출력할
출력할 수 있음. (cleanup phases)

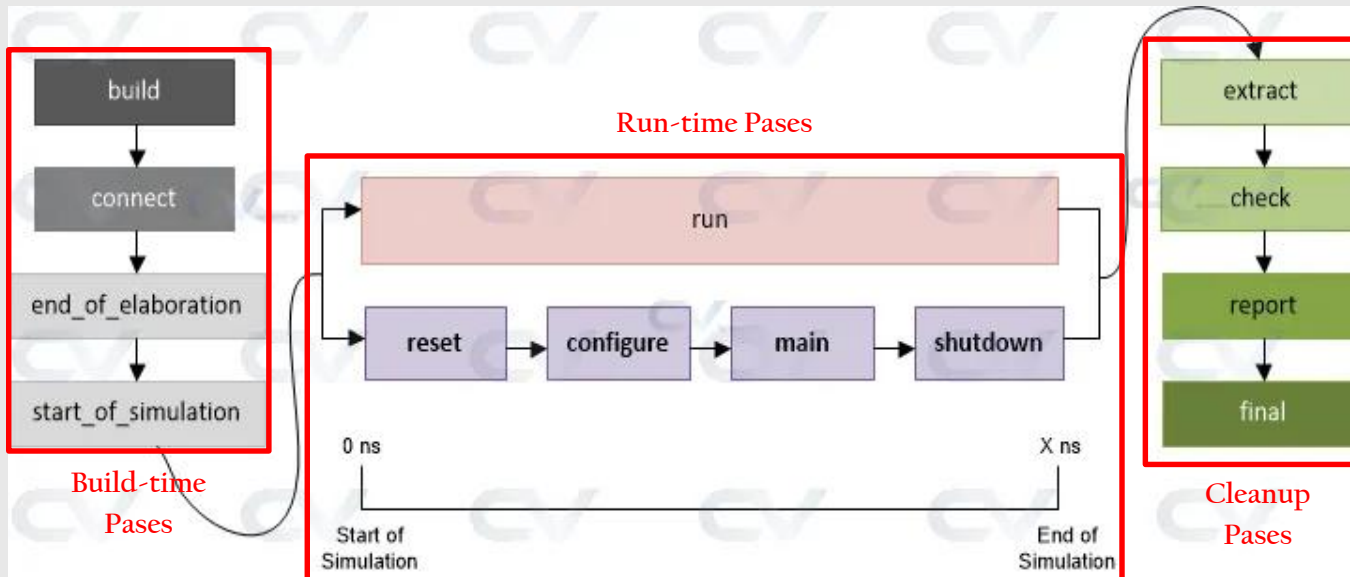
→ phase는 testbench 전체의 동작 순서를 동기화(Synchronization)하는 메커니즘.

● Then Why We Need Synchronization...

- 각 Component에 대한 생성 순서, 동작 타이밍, 종료 조건이 제각각인데,
Component 수가 늘어날수록 이에 대한 처리가 복잡해짐. (subsystem/SoC-level)
- 따라서 누가 언제 시작/종료하는지 관리하는 메커니즘이 필요한데,
이것은 code-level에서 강제하는 자치가 phase + objection

2) Phase Grouping & Overview

- 3 Groups of phases



Source: ChipVerify

1. Build-time Phases

- Component Tree를 만들고 연결하는 단계
- function base method
- Top-down Create (`build_phase()`)
- Bottom-up Connect (`connect_phase()`)

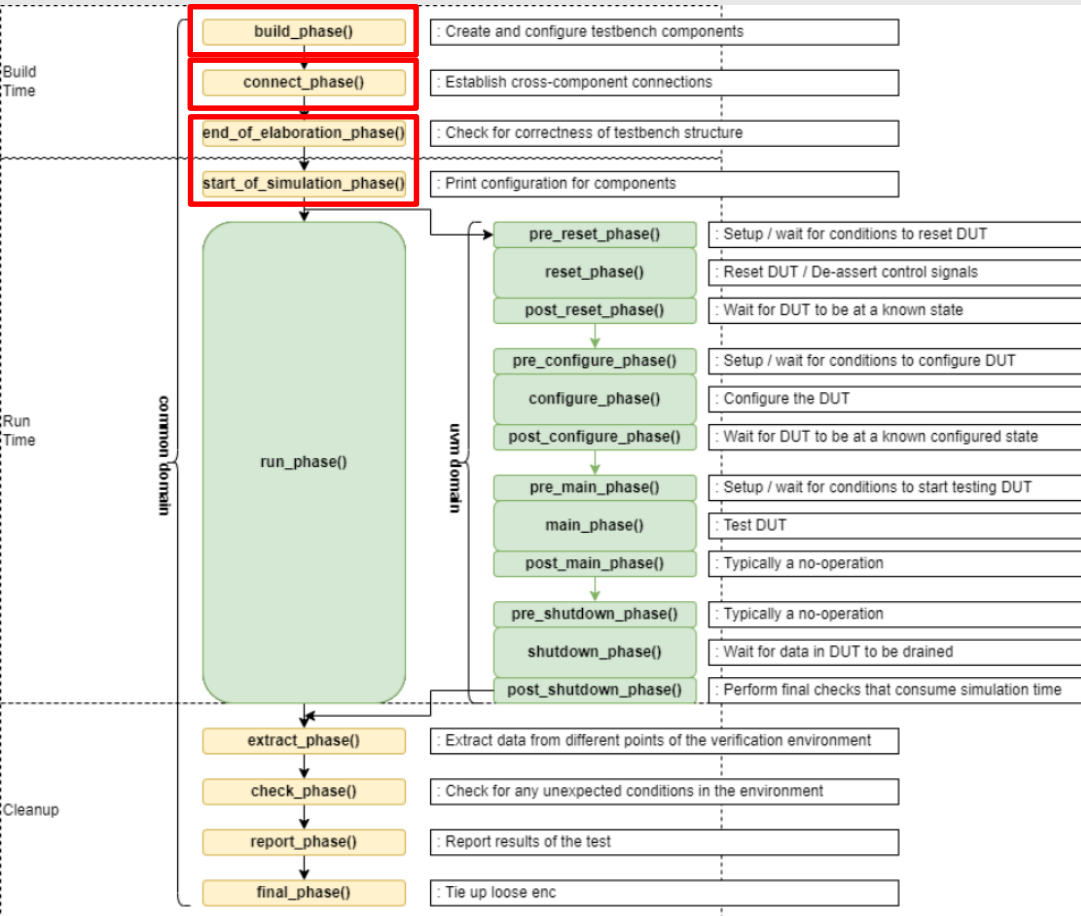
2. Run-time Phases

- stimulus/response가 이동하며 실제 시뮬레이션 시간이 흐르는 단계
- task base method (simulation time 제어 필요)
- Parallel Run

3. Cleanup Phases

- 결과를 모아서 검증/보고하는 단계
- function base method
- Bottom-up Run

3) Build-time Phases in Detail



• build_phase()

- Component Tree를 생성하는 단계 (::type_id::create())
- config DB의 set()/get()을 통해 config 값을 하위로 전달

• connect_phase()

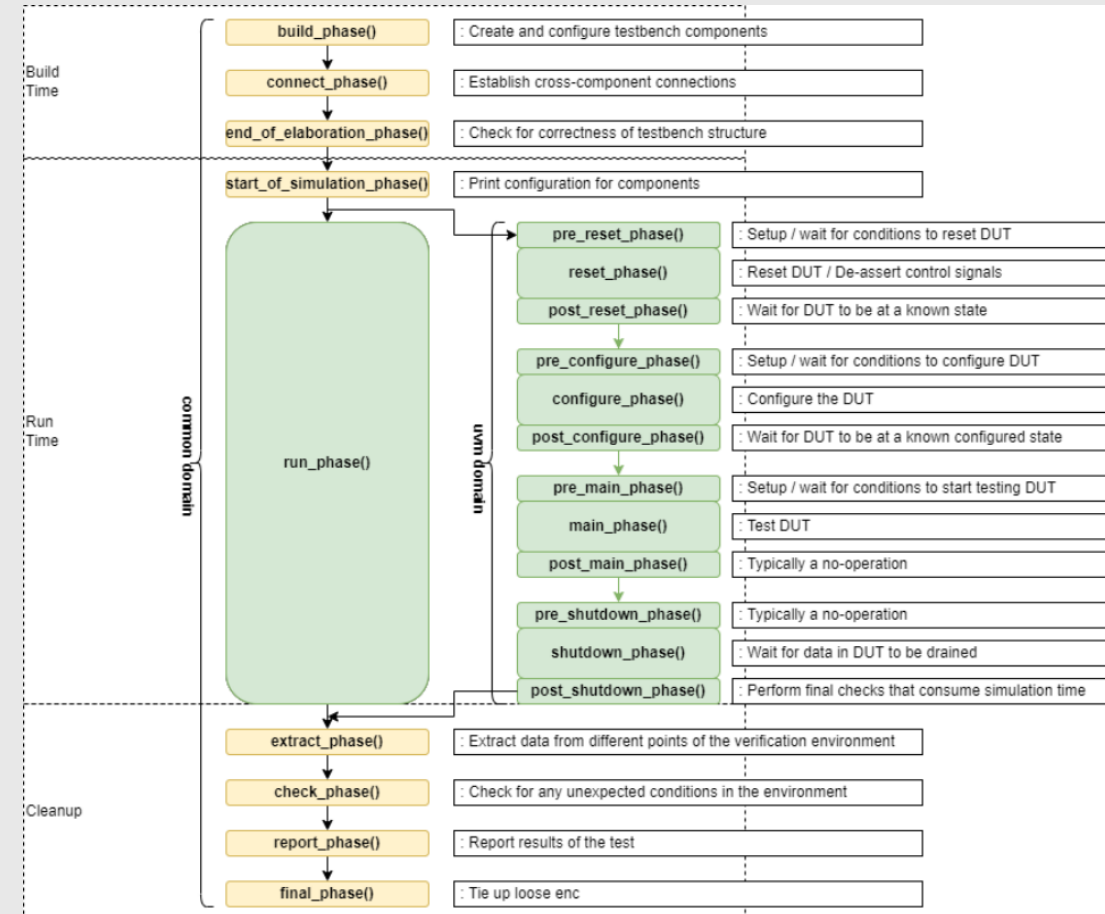
- 이미 생성된 Component 사이를 TLM port/export/imp로 연결하는 단계
- 이 과정에서 Transaction 간의 path가 완성됨.

• end_of_elaboration_phase(), start_of_simulation_phase()

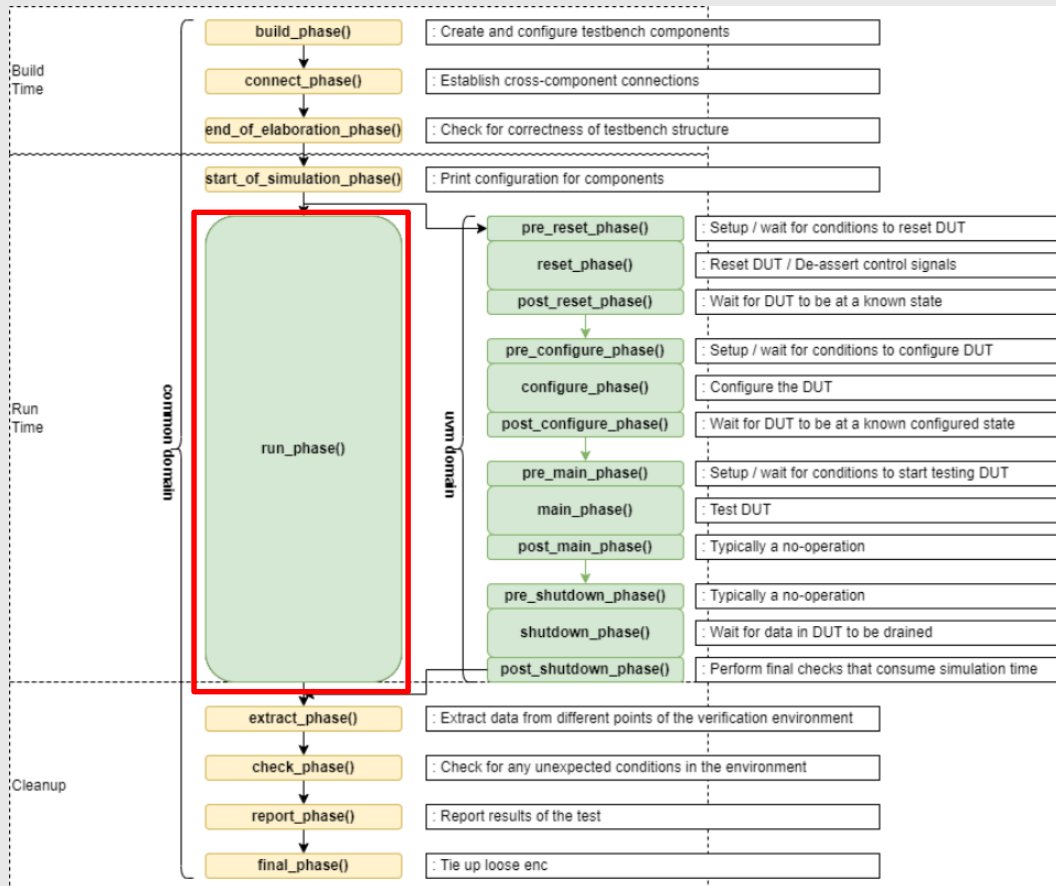
- Testbench 구조를 검증하거나, run-time phase 직전의 마지막 준비 단계
- 주로 uvm_top.print_topology()로 Hierarchy 구조와 config 확인 기능 수행

3) Build-time Phases in Detail

- **build_phase()**
 - Component Tree를 생성하는 단계
(::type_id::create())
 - config DB의 set()/get()을 통해 config 값을 하위로 전달
- **connect_phase()**
 - 이미 생성된 Component 사이를 TLM port/export/imp로 연결하는 단계
 - 이 과정에서 Transaction 간의 path가 완성됨.
- **end_of_elaboration_phase(), start_of_simulation_phase()**
 - Testbench 구조를 검증하거나, run-time phase 직전의 마지막 준비 단계
 - 주로 uvm_top.print_topology()로 Hierarchy 구조와 config 확인 기능 수행



4) Run-time Phase & Objection



• run_phase()

- 실제 시뮬레이션이 진행되며 stimulus가 이동하는 단계
- 각 Component (driver/monitor/scoreboard/sequence)의 task Method가 Parallel 하게 실행
- 필요에 따라 pre_reset/reset/main/shutdown 등 세분화된 run-time phases로 나누어 사용할 수 있음.

• Objection Mechanism

- Phase의 종료를 판단하는 기준
- `phase.raise_objection(this)`: phase 종료 지연
- `phase.drop_objection(this)`: phase 종료 요청
- 해당 phase에 대한 objection count가 0이 되면 그 phase는 종료.

5) Discussion

● 왜 `build_phase()` 만 top-down으로 실행될까?

- 1. tb환경은 nested-structure 이고 하위 Component는 상위 Component가 먼저 정의되어야 하기 때문에, 호출 순서는 필연적으로 Top-down으로 실행됨.
- 2. Config DB 설정을 위해서도 마찬가지. 상위 Component (test/env)가 config DB에 값을 set 하고, 그 다음 하위 Component (agent/driver/monitor)가 build_phase에서 get 하므로 Top-down이 적절함.
- 3. Factory override와 계층 이름 안정성을 위해
 - Factory를 사용해 type/instance override 할 때, instance 경로 기반으로 (`uvm_test.env.agent.driver` 같은) override를 하는 경우가 많음.
 - 그러기 위해서는 먼저 Parent 쪽에서 어떤 이름/구조로 Child를 만들지 결정해야 하고, 그 이름/구조에 맞춰서 override 규칙이 적용됨.
 - 이때 상위 컴포넌트가 이 규칙을 결정해야 override가 안정적으로 작동함.
- 4. 나머지 phase는 “Component Tree 가 이미 완성된 후” 라서 bottom-up/parallel로 호출해도 문제 없음.



2. **UVM Factory**

02

1) UVM Factory

● What is Factory ?

- TB 설계를 더 유연하고 확장 가능하게 만들어주는 메커니즘
- 등록, 생성, override 세 단계로 나뉘어짐

● Why Factory?

- UVM 및 SV에서 class object를 생성하는 방법 : new()
- ex)
 - 현재 TB가 v1의 data packet, TB Component을 사용중이라 가정
 - v2가 released가 되어서 새로운 pattern의 data packet을 사용해야함
 - TB상의 코드에 이 새로운 packet을 사용하기 위해 많은 부분을 수정해야함
 - UVM에서는 factory를 이용해 instance를 수정하지 않고 새로운 버전으로 적용가능
 - UVM에서는 create() method를 활용해 class object를 생성하는게 일반적

2) Override의 개념

● 상황 1.

```
class monitor extends uvm_monitor;
...;
virtual task run_phase(uvm_phase phase);
    forever begin
        packet pkt;
        pkt = new("pkt");
        get_packet(pkt);
    end
endtask
endclass
```

// 이렇게 새로이 정의된 packet을 코드 수정없이 monitor에 어떻게 적용할 것인가

```
class bad_packet extends packet;
...;
bit bad;
virtual function int compute_crc();
endclass
```

-> monitor의 transaction을
새로 작성한 것으로 변경하는 case

● 상황 2.

```
class input_agent extends uvm_agent;
    packet_sequencer sqr;
    driver drv;
...;
virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    sqr = new("sqr", this);
    drv = new("drv", this);
...;
endfunction
endclass
```

// 이렇게 새로이 정의된 driver를 기존 code수정없이 어떻게 적용할지

```
class NewDriver extends driver;
    virtual task run_phase(uvm_phase phase);
        if (dut.vif.done == 1)
            ...;
    endtask
endclass
```

-> agent에서 기존 driver를
새로 작성한 것으로 변경하는 case

- UVM factory를 이용해 기존 클래스를 다른 클래스로 '대체(substitute)'.
- TB 코드를 직접 수정하지 않고도 Test 환경의 동작을 변경, 유연성 확보 가능
- Override의 종류 :
 - Type override - 특정 class의 타입을 통째로 클래스 타입으로 교체
 - Instance override — 특정 계층 경로에서 생성되는 객체에 대해서만 override를 적용

3) Factory Override 예시

- base driver

```
// 기존 driver
class uart_driver extends uvm_driver #(uart_item);
  `uvm_component_utils(uart_driver)

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual task run_phase(uvm_phase phase);
    forever begin
      seq_item_port.get_next_item(req);
      `uvm_info("DRV", $sformatf("TX DATA = %0h", req.data), UVM_MEDIUM)
      seq_item_port.item_done();
    end
  endtask
endclass
```

- uvm_component_utils : factory 등록
-> create() 될 수 있는 타입이 됨
- (test에서) override가 걸리면
이 클래스는 생성되지 않게 되고,
override된 클래스가 대신 생성됨

- override 할 새로운 driver

```
class uart_driver_delay extends uart_driver;
  `uvm_component_utils(uart_driver_delay)

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual task run_phase(uvm_phase phase);
    forever begin
      seq_item_port.get_next_item(req);
      #(req.delay_cycles); // delay injection
      `uvm_info("DRV_DELAY",
        $sformatf("Delayed TX DATA = %0h", req.data),
        UVM_MEDIUM)
      seq_item_port.item_done();
    end
  endtask
endclass
```

- base driver인 uart_driver를 상속
- 마찬가지로 factory에 등록
- test에서 override 설정을 하면 factory는
base(driver)가 아니라
이 driver_delay를 생성하도록 바뀜.

3) Factory Override 예시

- agent.sv

```
class uart_agent extends uvm_agent;
  `uvm_component_utils(uart_agent)

  uart_driver drv;
  uvm_sequencer #(uart_item) seqr;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    seqr = uvm_sequencer#(uart_item)::type_id::create("seqr", this);

    // * override의 핵심 포인트: 여기서 driver가 factory 경유로 생성됨
    drv = uart_driver::type_id::create("drv", this);
    // † base type으로 create하는데, override가 걸리면 delay driver가 생성됨
  endfunction
endclass
```

- driver를 포함하는 agent에서 factory 경유로 생성
- driver는 반드시 type_id::create로 생성
- create()가 실행되면 factory는:
 - (test에) override가 있는지 확인
 - override된 타입이 있다면 그 타입으로 객체를 생성
 - override가 없다면 기본 uart_driver 생성
- create()는 그냥 객체 생성이 아니라,
factory에게 '어떤 타입을 생성할지 물어보는 시스템'

3) Factory Override 예시

- delay_test.sv

```
class uart_delay_test extends uvm_test;
  `uvm_component_utils(uart_delay_test)

  uart_env env;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);

    // * override 선언부 - 이 test에서는 driver를 delay-driver로 교체
    uvm_factory::get().set_type_override_by_type(
      uart_driver::get_type(),
      uart_driver_delay::get_type()
    );

    env = uart_env::type_id::create("env", this);
  endfunction
endclass
```

- 기존 base test가 아닌,
새롭게 설계한 delay_driver를 override하기 위한 새로운 Test.
- 이 부분이 override 시스템의 핵심
 - override는 test에서 선언
 - agent에서 driver가 create()될 때,
test로부터 factory가 override 규칙을 확인하고,
규칙에 따라 uart_driver_delay를 생성.
- 즉 override는:
 - 선언은 test에서, 적용은 agent의 build_phase에서
 - 생성은 factory가 create 호출 시점에서
 - 효과는 driver 동작에서 나타남



3.

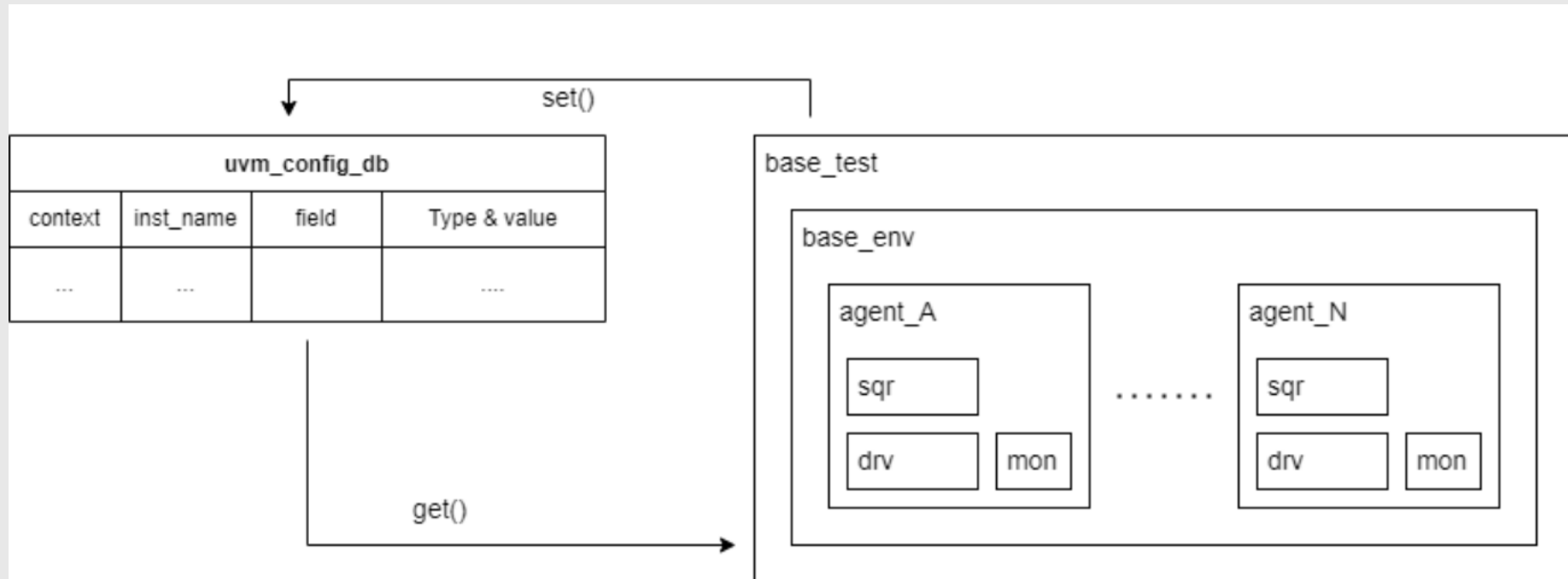
UVM Config

03

1) UVM Config

●What is UVM Config?

- 테스트벤치 구성(파라미터, 핸들, virtual interface 등)을 컴포넌트 계층으로 전달하기 위한 **전역 키-값 저장소**(global centralized DB).



1) UVM Config

●API 구조

```
uvm_config_db #(type)::set( context, inst_name, field, value) ;  
uvm_config_db #(type)::get( context, inst_name, field, value) ;
```

#(type) : 템플릿 타입 결정

-> 저장(set)/검색(get)되는 값의 type을 결정

-> ex) #(virtual alpha_interface)

set/get : config db에 대해 설정/가져오는 동작을 결정

1) UVM Config

- API 구조

```
uvm_config_db#(type)::set( context, inst_name, field, value) ;  
uvm_config_db#(type)::get( context, inst_name, field, value) ;
```

context:

- 의미: 설정 항목을 "어디에 붙여둘지" 지정하는, uvm_component 핸들
- 동작:
 - set()의 context : 해당 context 컴포넌트의 config DB에 항목을 저장
 - get()의 context : requester가 상위로 올라가며 각 context의 config DB 에서, 설정한 inst_name을 바탕으로 일치하는 항목을 검색

inst_name:

- 의미 : 어떤 컴포넌트에 적용할 것인지 지정하는 문자열
- ex) : " * " : 모든 컴포넌트에 적용하겠다(와일드카드)

1) UVM Config

- **set()** :

- 보통 top-module(_tb)에서 수행.
- 컴포넌트들이 각각의 build_phase에서 get으로 읽을 수 있게 setting

```
module apb_alpha_tb;
  ...
  initial begin
    ...
    uvm_config_db#(virtual apb_alpha_interface)::set(uvm_root::get(), "*", "vif", apb_if );
    ...
  end
  ...
endmodule
```

1. 루트 컨텍스트(`uvm_root::get()`) 에 " *" 라는 인스턴스 이름으로 vif 키에 apb_if 값을 등록

1) UVM Config

• get() :

- 각 component들의 build_phase()에서 실행
- 호출하는 컴포넌트(보통 this)를 시작점으로 상위 계층을 검색하면서 등록된 inst_name 패턴과 매칭되는 항목을 찾음.

```
class apb_alpha_driver extends uvm_driver #(apb_alpha_transaction);  
    ...  
  
    virtual apb_alpha_interface vif;  
  
    function void build_phase(uvm_phase phase);  
        super.build_phase(phase);  
        if (!uvm_config_db#(virtual apb_alpha_interface)::get(this, "", "vif", vif))  
            `uvm_fatal("NOVIF", "Virtual interface not set for Alpha Driver")  
    endfunction
```

1. this(driver) 컨텍스트에서 호출.
2. 빈 inst_name("")으로 조회 요청
3. 상위 context(루트)에서 설정된 "*" 패턴이 매칭되어 vif를 수신

1) UVM Config

● 실제 코드 예시

```
module apb_alpha_tb;
...
initial begin
...
    uvm_config_db#(virtual apb_alpha_interface)::set(uvm_root::get(), "*", "vif", apb_if );
...
end
...
endmodule
```

```
class apb_alpha_driver extends uvm_driver #(apb_alpha_transaction);
...

virtual apb_alpha_interface vif;

function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if (!uvm_config_db#(virtual apb_alpha_interface)::get(this, "", "vif", vif))
        `uvm_fatal("NOVIF", "Virtual interface not set for Alpha Driver")
endfunction
```

1. **get(this, "",...)** -> driver에서 시작해 evn -> test 루트로 올라가며 각 context의 config DB에 등록된 inst_name 패턴(ex) " * ") 등과 this(driver)의 실제 경로를 매칭
2. **set(uvm_root::get(), "*"...)**이 루트에 있기 때문에, wildcard와 매칭되어 vif를 결국 받게 됨



4.

UVM

Macro, Methods

04

1) Print & Reporting Macros

- `$display` 대신 UVM Reporting Macro를 사용하는 이유?

세부적인 메시지 필터링, 파일 출력, 시뮬레이션 중단 레벨 조정 가능.

Macros	Description
<code>`uvm_info(ID, MSG, VERBOSITY)</code>	로그 메시지 출력 + verbosity를 parameter로 받아 메시지 필터링
<code>`uvm_warning(ID, MSG)</code>	경고 메시지 출력 + 시뮬레이션 계속 진행
<code>`uvm_error(ID, MSG)</code>	오류 발생 메시지 출력 + 시뮬레이션 계속 진행
<code>`uvm_fatal(ID, MSG)</code>	오류 발생 메시지 출력 + 시뮬레이션 종료

- VERBOSITY Level Enumeration in Reference Implementation (UVM IEEE 1800.2)

```
382 typedef enum
1 {
2     UVM_NONE = 0,
3     UVM_LOW = 100,
4     UVM_MEDIUM = 200,
5     UVM_HIGH = 300,
6     UVM_FULL = 400,
7     UVM_DEBUG = 500
8 } uvm_verbosity;
^
0 c:\Users\hsyoon_ser9\Downloads\UVM-1800.2-2020.3.1\1800.2-2020.3.1\src\base\uvmm_object_globals.svh -- NORM
```


2) Sequence Macros

- ``uvm_do`: sequence/sequence_item 객체를 생성하고 Randomize 한 뒤, 현재 sequencer (m_sequencer) 에서 실행.
 - ``uvm_do` 계열의 매크로를 사용하면 컴파일러는 아래와 같은 과정을 자동으로 코드로 확장함.
 - Creation: `type_id::create()` 를 호출하여 factory로 객체 생성.
(``uvm_do(req)` 에서 `req == null` 일 경우)
 - Wait for grant: `start_item()` 을 호출하여 Driver가 준비될 때까지 대기.
 - Randomization: `randomize()` 를 호출 (`with` 가 있다면 constraint 적용)
 - Execute: `finish_item()` 을 호출하여 Driver로 전송.
- ``uvm_do_with`: ``uvm_do` + Constraints 기능 추가
- ``uvm_do_on_with`: ``uvm_do_with` + 객체를 실행할 특정 Sequencer를 명시적으로 지정
- NOTE: UVM 1.2 당시에는 SystemVerilog 매크로가 Default Argument를 제대로 지원하지 않아 기능별로 따로 만들어야 했지만,
UVM IEEE 1800.2 부터는 ``uvm_do` 하나의 매크로로 통합해서 처리할 수 있게 되었음.

3) Factory Registration of Component & Object

- Purpose of Registration Macros

Factory 기반 생성, type override를 사용하기 위해서는 모든 UVM class를 Factory에 등록해야 함.

- Basic Macros

- ``uvm_component_utils(TYPE)`

- `get_type_name()`, `create()`, `get_object_type()` 등 자동 생성

- ``uvm_object_utils(TYPE)`

- `uvm_object` / `uvm_sequence_item` 등 “데이터 객체” 등록

- NOTE: 다음과 같이 field macro와 함께 Extended Form으로도 사용 할 수 있음. (Cont'd)

```
`uvm_component_utils_begin(TYPE)
    `uvm_field_int(data, UVM_ALL_ON)
    <...>
`uvm_component_utils_end
```

4) Field Automation

- Purpose of Field Macros

멤버 변수들을 Factory에 자동 등록하여 transaction/sequence_item에 대한 공통 동작을 자동화

- 즉, `copy()/compare()/print()/record()/pack()/unpack()` 등의 동작을 수동으로 구현할 필요 없이 Macro 한 줄로 처리 가능

- Field Macros Example

- ``uvm_field_int(field, flag)`
- ``uvm_field_enum(type, field, flag)`
- ``uvm_field_object(field, flag)`
- ``uvm_field_array_int(field, flag)`

flag: 이전에서 parameter로 준 field에 대해 어떤 기능을 (`copy/compare/print/pack..`) 자동화 할지 지정하는 옵션

5) UVM Methods

- Factory & Construction
 - `TYPE::type_id::create("name", parent)`
 - Factory를 통해 Component/Object 생성
 - type override가 적용되는 지점
 - `get_type_name()`, `get_full_name()`
 - Logging/Debugging 시 class 이름 + hierarchical path 출력
- Configuration & Topology
 - `uvm_config_db#(T)::set(this, "env.agent", "vif", vif);`
`uvm_config_db#(T)::set(this, "", "vif", vif);`
 - virtual interface, parameter, mode 등을 상/하위로 전달
 - `uvm_top.print_topology()`
 - build/connect 이후 전체 TB 구조와 인스턴스 이름 확인
- Reporting & Control
 - `set_report_verbosity_level()`, `..verbosity_level_hier()`, `..severity_action();`
 - 특정 Component 또는 하위 계층에 대한 로그 수준/동작 제어

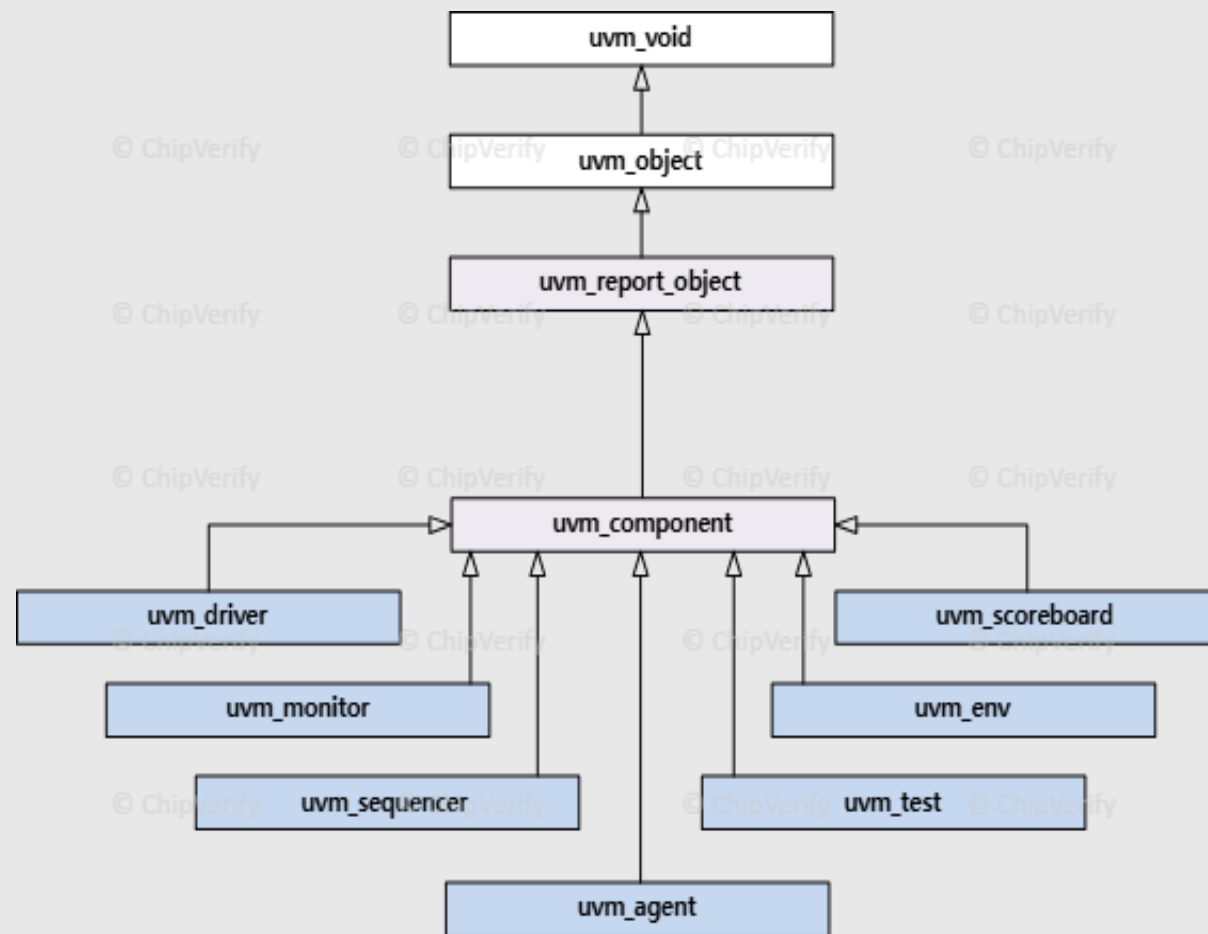


5. **UVM Component**

05

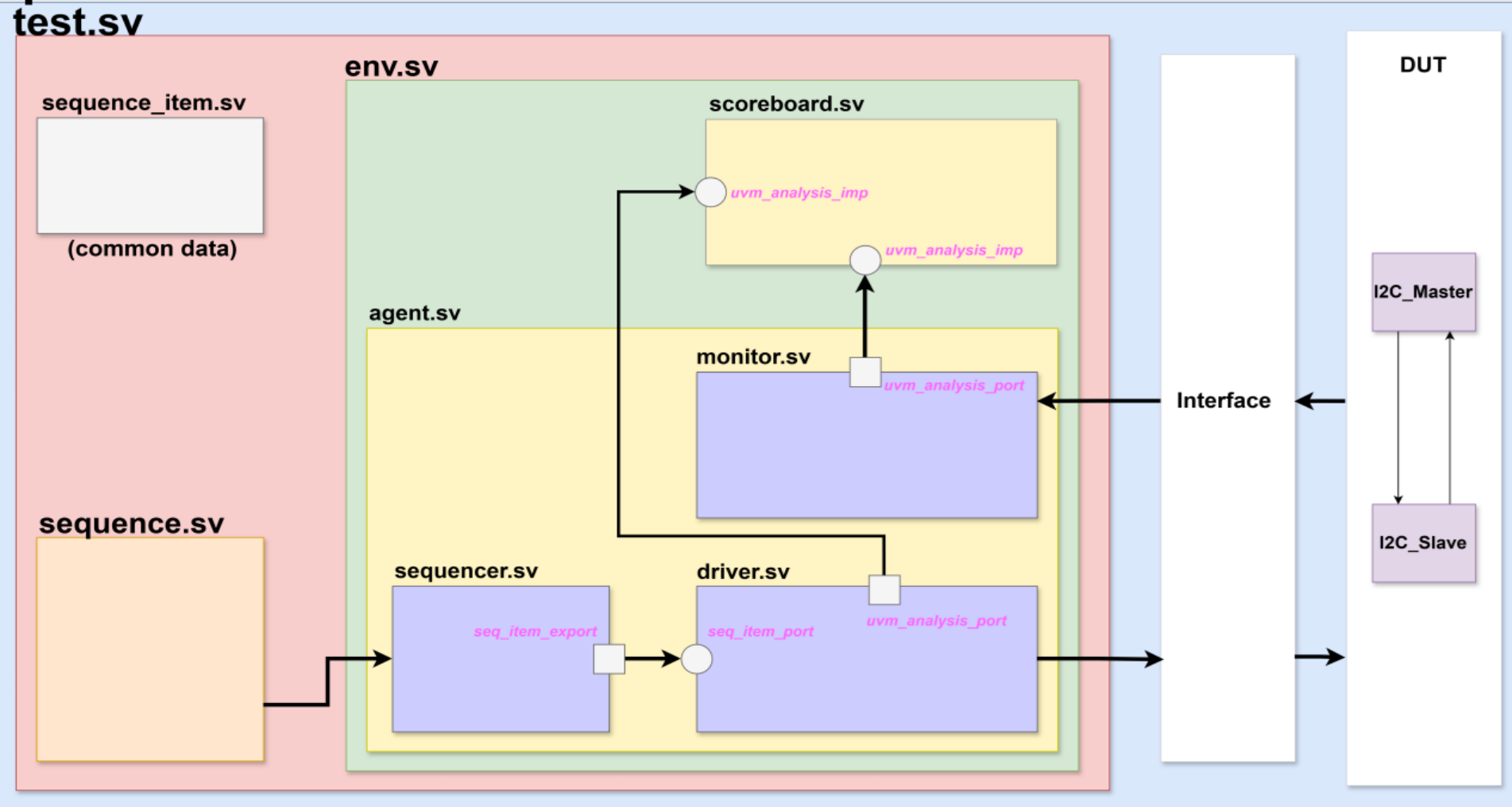
1) Concept of Component

- Concept:
UVM testbench 구조의 핵심 빌딩 블록
- 특징
 - 계층 구조
 - 자동 등록
 - 고정 수명 (Phase 기반 실행)
- 목적
 - 구조화된 환경 구축
 - 재사용성 극대화
 - 표준화된 동작
 - 구성 용이성



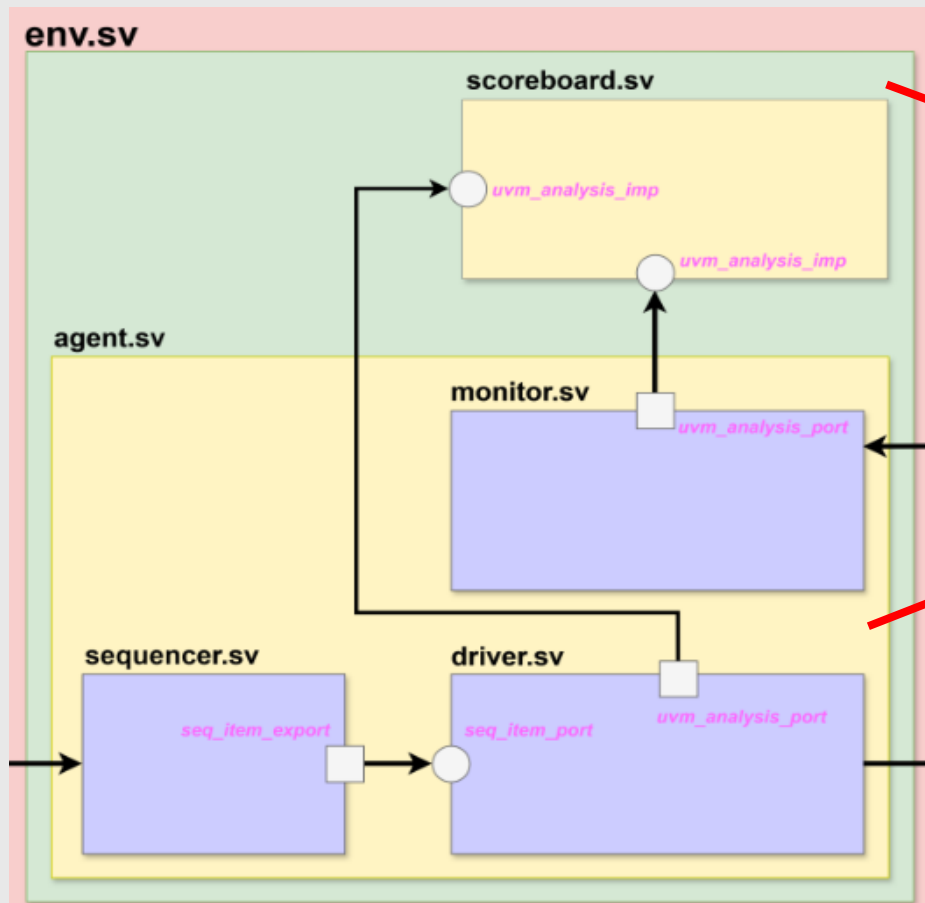
2) Main Component Overview

top



3) Component Details (1)

- env: 하나 이상의 agent, scoreboard, coverage collector를 포함하는 상위 레벨 컨테이너
- agent: 특정 인터페이스/프로토콜(AXI, APB, custom bus 등)을 캡슐화하여 처리하는 단위



```
class a_environment extends uvm_env;
    `uvm_component_utils(a_environment)
    function new(input string name = "ENV", uvm_component c); super.new(name, c);
    endfunction
```

```
a_agent a_agt;
a_scoreboard a_scb;
a_logger a_log;
```

```
function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    a_agt = a_agent::type_id::create("AGT", this);
    a_scb = a_scoreboard::type_id::create("SCB", this);
    a_log = a_logger::type_id::create("LOG", this);
endfunction
```

다른 환경 및 검증 구성 요소
선언 및 구축

```
function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    a_agt.a_mon.send.connect(a_scb.recv);
    a_agt.a_mon.send.connect(a_log.analysis_export);
endfunction
endclass
```

Virtual Sequence Start

```
class a_agent extends uvm_agent;
    `uvm_component_utils(a_agent)
    function new(input string name = "AGT", uvm_component c); super.new(name, c);
    endfunction
```

```
a_monitor a_mon; a_driver a_drv; uvm_sequencer #(a_seq_item) a_sqr;
function void build_phase(uvm_phase phase);
    super.build_phase(phase); a_mon = a_monitor::type_id::create("MON", this);
a_drv = a_driver::type_id::create("DRV", this);
    a_sqr = uvm_sequencer #(a_seq_item)::type_id::create("SQR", this);
endfunction
```

다른 환경 및 검증
구성 요소 선언 및
구축

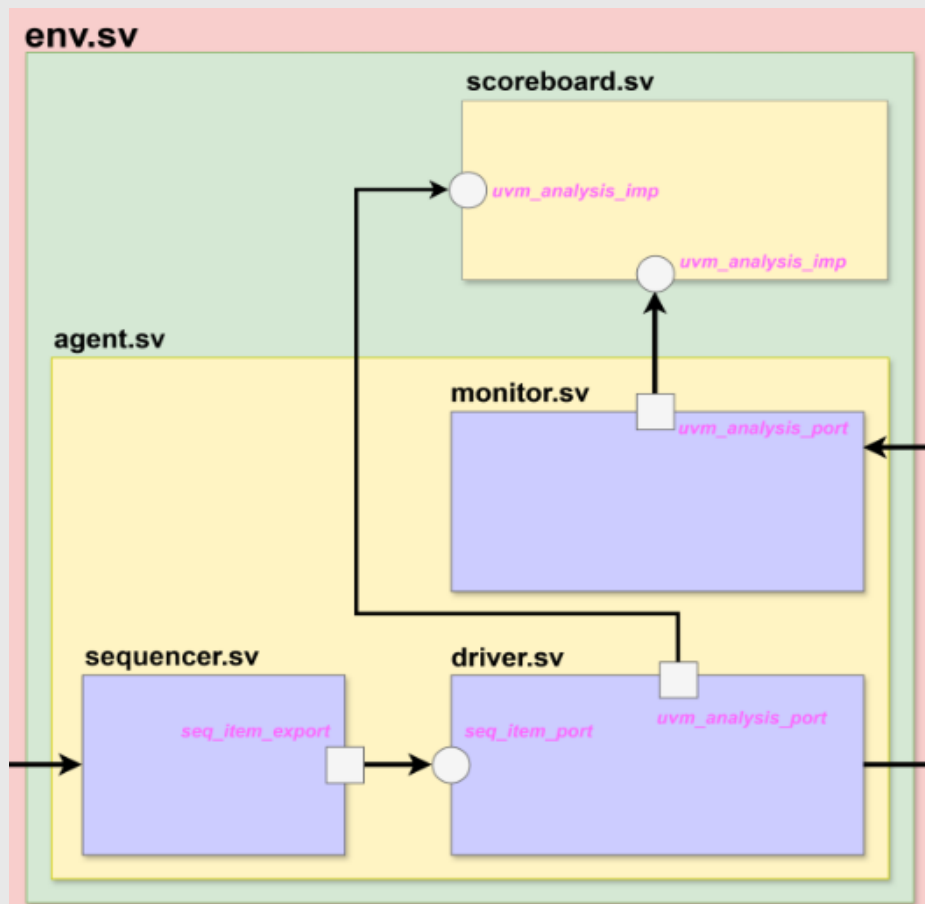
```
function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
a_drv.seq_item_port.connect(a_sqr.seq_item_export);
endfunction
endclass
```

Virtual Sequence Start

Driver & Sequencer Connect: Item 흐름을 설정

3) Component Details (2)

- sequencer: sequence에서 생성한 transaction 흐름을 관리, driver에 전달하는 중재자(Arbitrator) 역할
- driver: sequencer로부터 transaction을 받아 DUT 인터페이스 신호로 변환하여 drive



```
class a_sequence extends uvm_sequence #(a_seq_item);
    `uvm_object_utils(a_sequence)
    function new(input string name = "SEQ"); super.new(name); endfunction
    task body();
        a_seq_item a_item;
        a_item = a_seq_item::type_id::create("ITEM");
        a_item.set_item_context(this);
        for (int i = 0; i < 10; i++) begin
            start_item(a_item);
            if(!a_item.randomize()) `uvm_error("SEQ","Randomize error");
            `uvm_info("SEQ", $sformatf("Data send to Driver a : %0d, b : %0d", a_item.a, a_item.b),
                UVM_LOW)
            finish_item(a_item);
        end
    endtask
endclass
```

Sequence Start

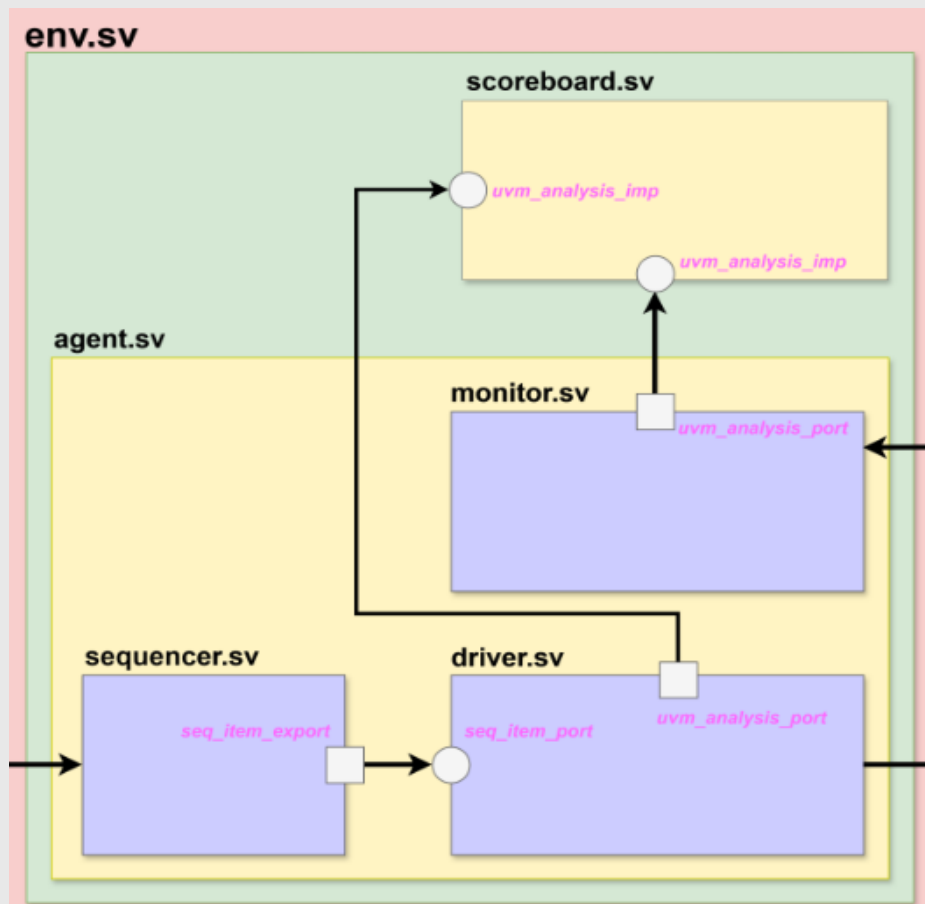
```
class a_driver extends uvm_driver #(a_seq_item);
    `uvm_component_utils(a_driver)
    function new(input string name = "DRV", uvm_component c); super.new(name , c); endfunction
    a_seq_item a_item; virtual adder_intf a_if;
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db#(virtual adder_intf)::get(this, "", "a_if", a_if)) begin `uvm_fatal("DRV",
            "Unable to access uvm_config_db"); end
    endfunction
    task run_phase(uvm_phase phase);
        forever begin seq_item_port.get_next_item(a_item); @(posedge a_if.clk); a_if.a <= a_item.a;
        a_if.b <= a_item.b;
        `uvm_info("DRV", $sformatf("Data send to Scoreboard a : %0d, b : %0d", a_item.a,
            a_item.b), UVM_NONE); @(posedge a_if.clk); seq_item_port.item_done(); end
    endtask
endclass
```

다른 환경 및 검증 구성 요소 선언 및 구축

Virtual Sequence Start

3) Component Details (3)

- monitor: DUT 인터페이스 신호를 transaction으로 변환하고 analysis port를 통해 다른 Component로 (scoreboard/coverage) 전달
- scoreboard: 예측(expected)과 실제(observed) Transaction을 비교하여 검증, pass/fail 여부 판단 및 보고



```
class a_monitor extends uvm_monitor;
    'uvm_component_utils(a_monitor)
    uvm_analysis_port #(a_seq_item) send;
    function new(input string name = "MON", uvm_component c); super.new(name, c); send =
    new("Write", this); endfunction
    a_seq_item a_item; virtual adder_intf a_if;
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db#(virtual adder_intf)::get(this, "", "a_if", a_if)) begin `uvm_fatal("MON",
        "Unable to access uvm_config_db"); end
    endfunction
    task run_phase(uvm_phase phase);
        forever begin @(posedge a_if.clk); a_item = a_seq_item::type_id::create("ITEM"); a_item.a =
        a_if.a; a_item.b = a_if.b; @(posedge a_if.clk); a_item.y = a_if.y;
        `uvm_info("MON", $sformatf("Data send to Scoreboard a : %0d, b : %0d, y : %0d", a_item.a,
        a_item.b, a_item.y), UVM_NONE); send.write(a_item); end
    endtask
endclass
```

다른 환경 및 검증 구
성 요소 선언 및 구축

Virtual Sequen
ce Start

```
class a_scoreboard extends uvm_scoreboard;
    'uvm_component_utils(a_scoreboard)
    uvm_analysis_imp #(a_seq_item, a_scoreboard) recv;
    function new(input string name = "SCB", uvm_component c); super.new(name, c); recv = new("Read",
    this); endfunction
    function void build_phase(uvm_phase phase); super.build_phase(phase); endfunction
    function void write(input a_seq_item item);
        `uvm_info("SCB", $sformatf("Data Received from Monitor a : %0d, b : %0d, y : %0d", item.a,
        item.b, item.y), UVM_NONE)
        if(item.y == (item.a + item.b)) begin `uvm_info("SCB", "PASSED", UVM_NONE) end
        else begin `uvm_error("SCB", $sformatf("FAILED: %0d + %0d != %0d", item.a, item.b, item.y))
        end
    endfunction
endclass
```

Receive를 위한 TLM Port
생성

다른 구성 요소와 연결
& 기존 Seq Item 요소
를 받아 Golden Model
생성 후 값을 받아 비교



6.

UVM

Communication Mechanism

06

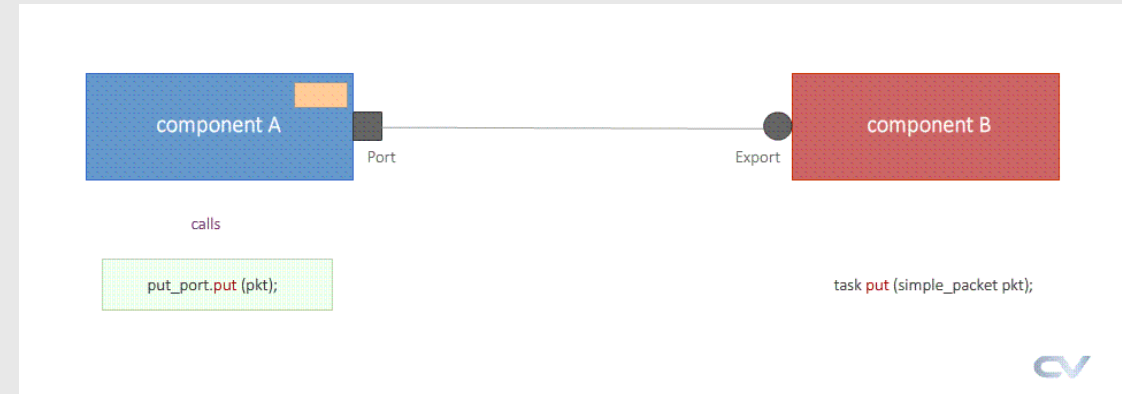
1) TLM (Transaction Level Modeling)

• 정의

- TLM Port는 UVM 환경 내의 component들이 transaction 단위로 통신하는 표준화된 방법을 제공
- HW Pin 레벨 신호 대신 sequence item과 같은 추상적 데이터 객체를 주고받음

• 사용 이유

- UVM 환경은 독립적인 component(driver, monitor, scoreboard 등)들의 집합으로 구성되어 있음
- 따라서 이들이 효율적이고 구조화된 방식으로 통신하는 것이 검증 환경의 성공에 매우 중요



```
class comp_a extends uvm_component {
  uvm_blocking_put_port #(int) send;

  rand int data;

  function new(string name = "COMP_A", uvm_component C);
    super.new(name, C);
    send = new("send", this);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
  endfunction

  function void start_of_simulation_phase(uvm_phase phase);
    super.start_of_simulation_phase(phase);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
    phase.raise_object(this);
    For (int i = 0; i < 10; i++) begin
      uvm_info("COMP_A", $format("int data = %0d", data), UVM_NONE);
      send.put(data);
    end
    phase.drop_object(this);
  endtask
endclass

class comp_b extends uvm_component {
  uvm_blocking_put_imp #(int, comp_b) recv;

  function new(string name = "COMP_B", uvm_component C);
    super.new(name, C);
    recv = new("recv", this);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
  endfunction

  function void connect_phase(uvm_phase phase);
    super.connect_phase(phase);
  endfunction

  function void start_of_simulation_phase(uvm_phase phase);
    super.start_of_simulation_phase(phase);
  endfunction

  task run_phase(uvm_phase phase);
    super.run_phase(phase);
  endtask

  task put (int data);
    uvm_info("COMP_B", $format("Received data = %0d", data), UVM_NONE);
  endtask
endclass
```

송신자 클래스 유형 포트

생성

해당 구성 요소나 환경내에

반드시 연결

Put을 통해 송신

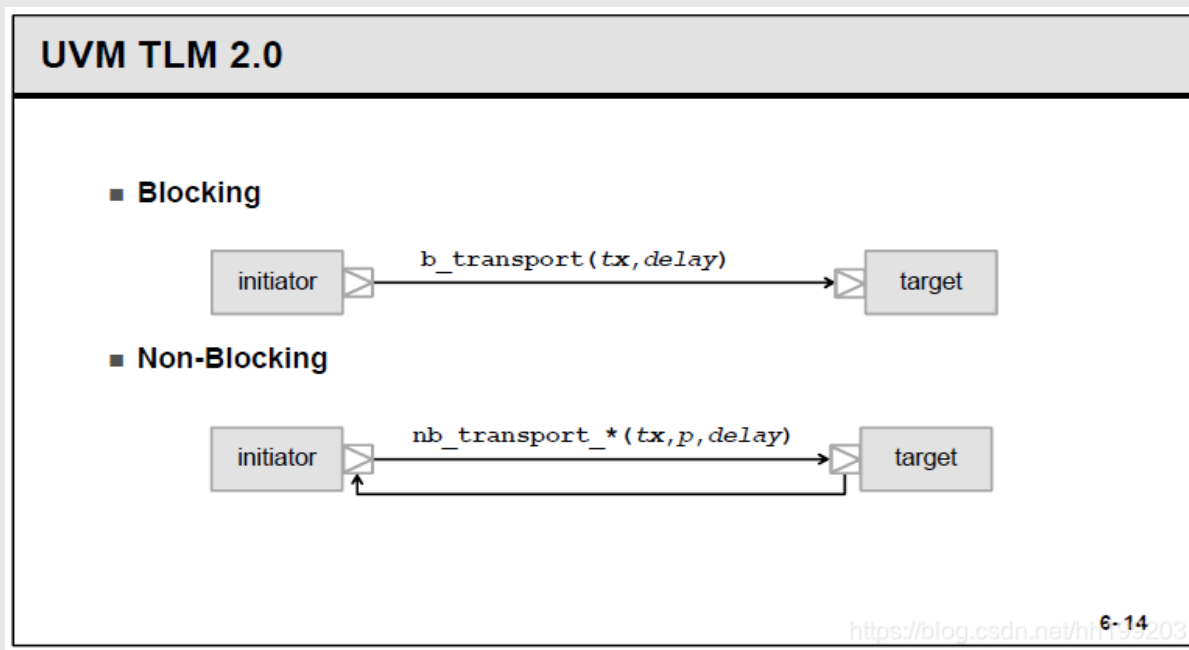
Put_imp를 통해 수신

Method를 구현하는 수신

2) 2-Type of TLM Interface

• Blocking or Non-Blocking

주로 component 간 TLM FIFO나 put / get / peek 포트를 사용하여 데이터를 전송할 때, Caller의 실행 흐름을 멈추게 할지 여부를 기준으로 구분



구분	Blocking Mode	Non-Blocking Mode
주요 목적	데이터 전송/수신 시 엄격한 동기화 보장	호출자 실행 흐름 유지를 위한 유연한 비동기식 데이터 교환
결과 반환	작업이 완료되거나 객체가 수신될 때까지 함수가 반환되지 않음.	작업 성공/실패 여부를 bit 값(1 또는 0)으로 즉시 반환함.
주요 사용처	TLM FIFO를 사용할 때, 데이터 전달의 정합성 보장이 최우선일 때.	버퍼가 가득 찼는지 확인하며 병렬 작업을 수행해야 할 때.
장점	코드가 단순하고, 데이터 전달 및 수신을 확실하게 보장함.	호출자가 대기하지 않고 다른 작업을 병행할 수 있어 유연성이 높음.
단점	데이터가 준비되지 않으면 데드락 또는 불필요한 대기 시간이 발생할 수 있음.	실패 시 재시도 또는 예외 처리로 직을 추가로 구현해야 함.

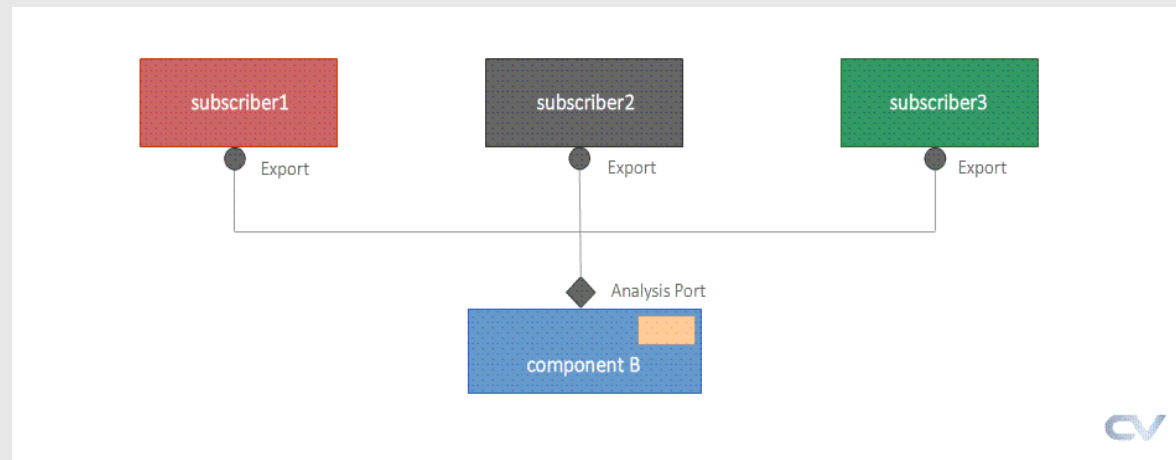
3) TLM Port Mode (1)

• Broadcast Mode

- 주로 Monitor가 DUT에서 캡처한 transaction을 여러 개의 component에게 Non-Blocking 방식으로 전파하기 위해 사용

• 방식

- One-to-Many : 하나의 발신자(monitor)가 여러 수신자(scoreboard, coverage, subscriber)에게 데이터 전송
- Non-Blocking : 발신자는 데이터를 전송한 후 수신자의 응답을 기다리지 않고 즉시 다음 작업 수행



```
virtual task run_phase (uvm_phase phase);
    super.run_phase (phase);

    for (int i = 0; i < 3; i++) begin
        simple_packet pkt = simple_packet::type_id::create ("pkt");
        pkt.id = i;
        void'(pkt.randomize());

        uvm_info(get_full_name(),
                 $sformatf("Broadcasting Packet ID: %0d, Data: %0h", pkt.id, pkt.data),
                 UVM_MEDIUM)

        // 브로드캐스트==
        ap.write (pkt);
        #10;
    end
endtask
```

표준 TLM_Port를 사용하여
1대1로 Random값 생성

```
virtual function void build_phase (uvm_phase phase);
    super.build_phase (phase);

    compB = componentB::type_id::create ("compB", this);
    // 매개변수화된 클래스에 #() 명시==
    sub1 = sub #()::type_id::create ("sub1", this);
    sub2 = sub #()::type_id::create ("sub2", this);
    sub3 = sub #()::type_id::create ("sub3", this);
endfunction

virtual function void connect_phase (uvm_phase phase);
    // 하나의 분석 포트(compB.ap)를 세 개의 Analysis Imps에 연결==
    compB.ap.connect (sub1.analysis_export);
    compB.ap.connect (sub2.analysis_export);
    compB.ap.connect (sub3.analysis_export);

    uvm_info(get_full_name(), "Analysis Port connected to three subscribers.", UVM_LOW)
endfunction
```

Environment에 Sub3개를
만들고 Component 연결

3) TLM Port Mode (2)

• PULL Mode

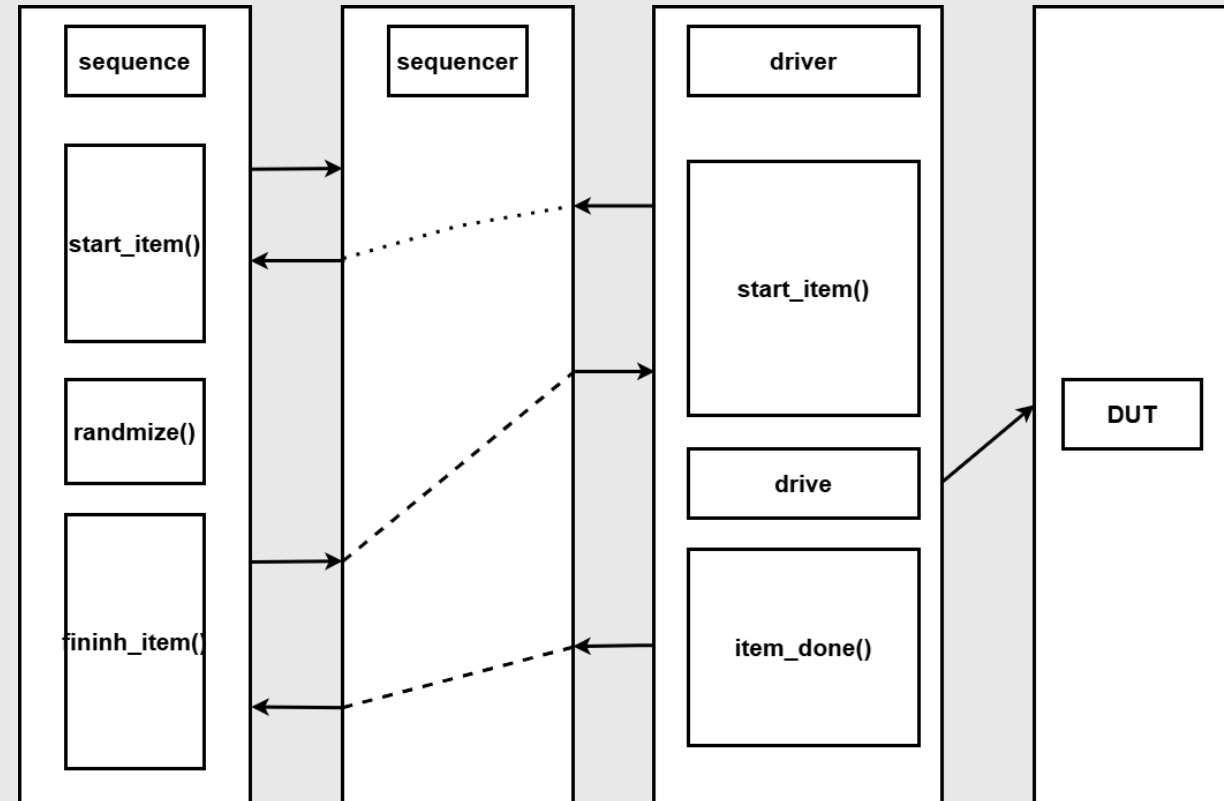
- Driver가 주도권을 가지고 데이터를 요청하여 Transaction을 가져가는 방식

• 특징

- Stimulus 전달 흐름을 효율적으로 제어 가능
(Driver가 DUT의 핀 레벨 타이밍에 묶여 있기 때문에, Driver가 준비될 때만 처리하도록 함으로써 성능 저하를 방지)

• 단계

- 시퀀스 생성
- driver 준비 및 요청
- Transaction 전달 및 driver 구동
- 핸드셰이크 완료 통보
- 시퀀스 해제



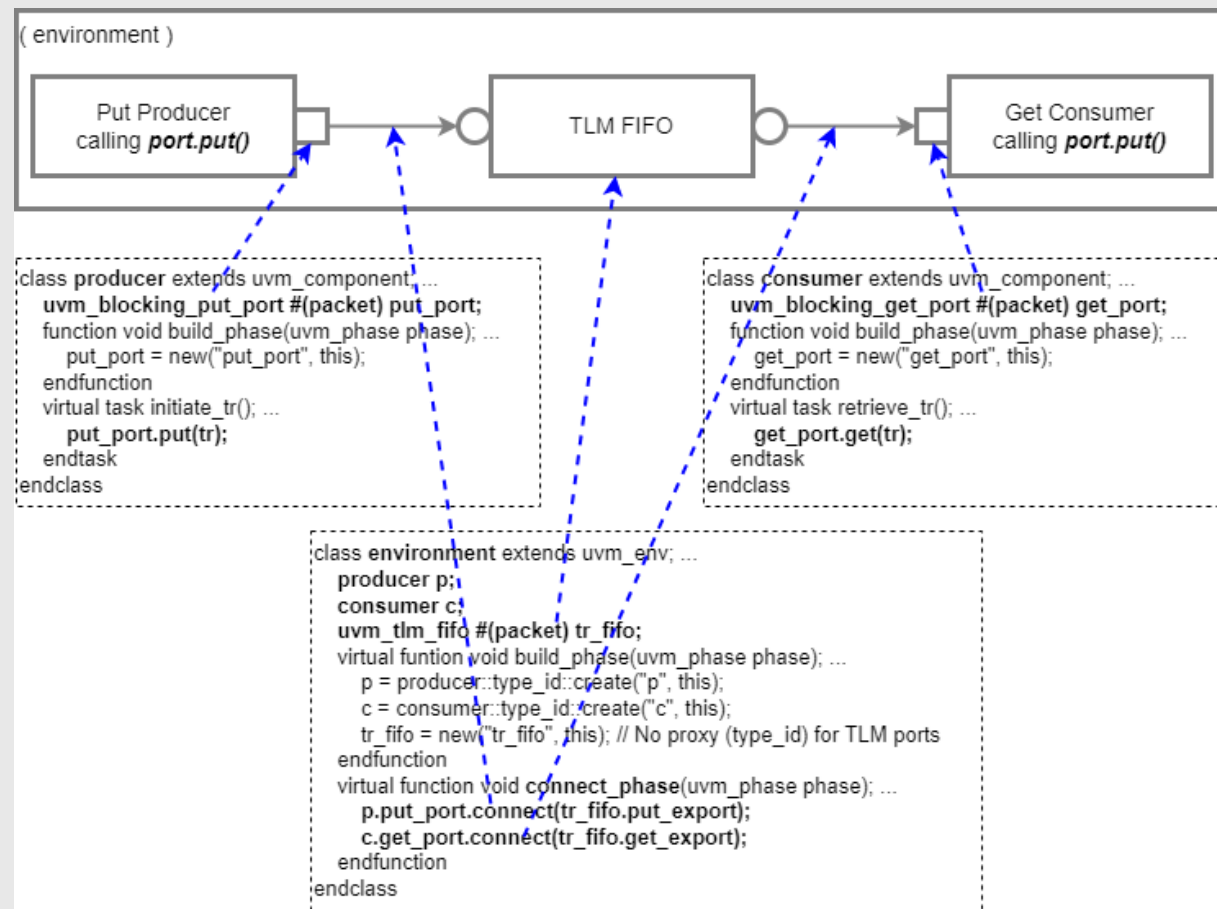
3) TLM Port Mode (3)

• PUSH Mode

- Producer component에서 port의 put()을 call하면서 transaction을 넘겨줌
- 연결된 connection을 따라 consumer componen에 있는 put()을 implement하여 넘겨진 transaction을 받고, 처리하는 형태

• 특징

- 데이터의 Producer가 데이터 전송 시점을 주도하고, Consumer에게 transaction item을 능동적으로 밀어 넣어(put) 전달
- 주로 TLM을 통해 component 간의 직접적인 데이터 흐름을 구현



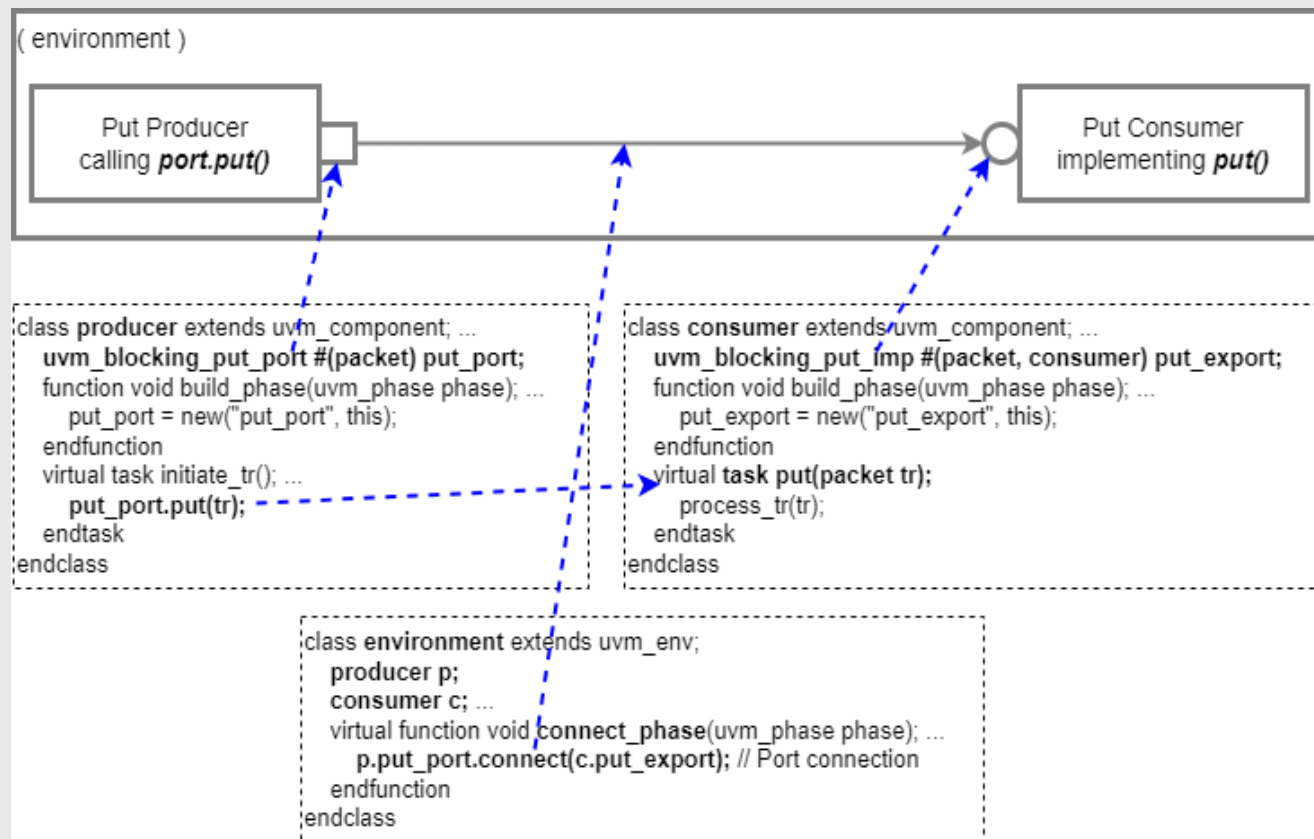
3) TLM Port Mode (4)

• FIFO Mode

- Producer와 Consumer간에 fifo가 있는 형태를 modeling하기 위해 사용

• 사용 이유

- 데이터를 생산하는 Producer와 Consumer 간의 타이밍을 분리(Decoupling)
- Transaction을 순서대로 (First-In, First-Out) 안전하게 저장 및 전달



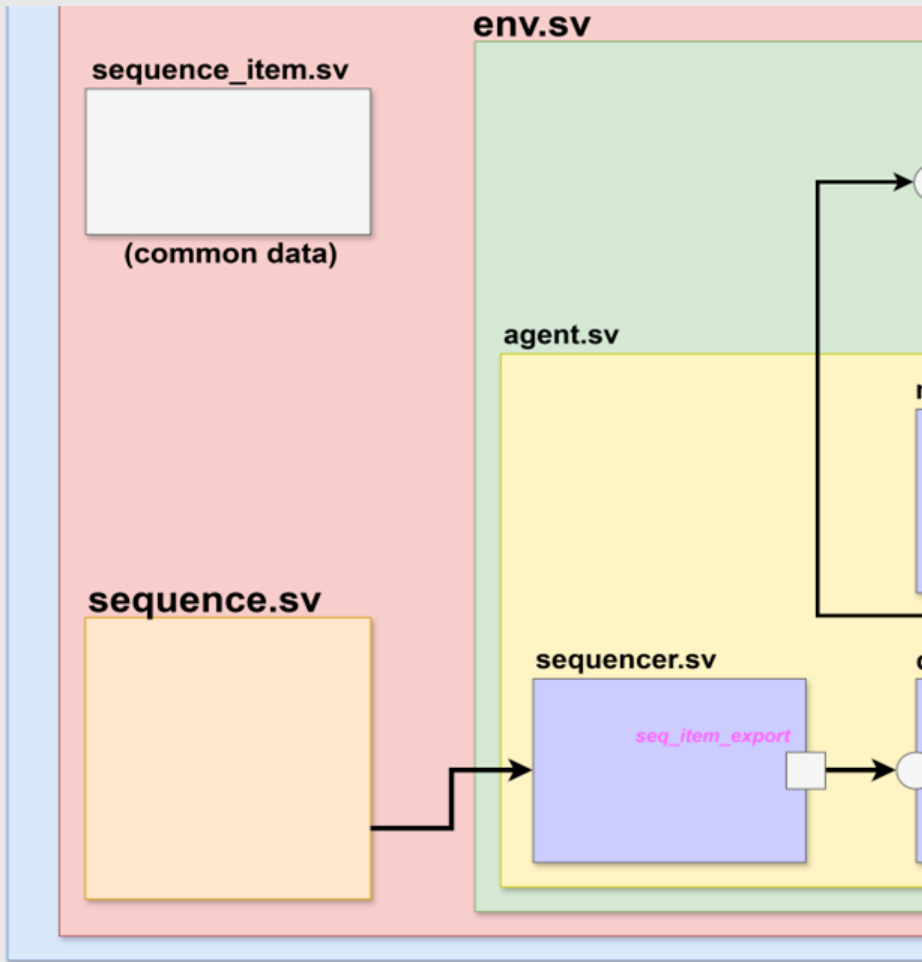


7. UVM Sequence

07

1) What is Sequence & Sequence Item

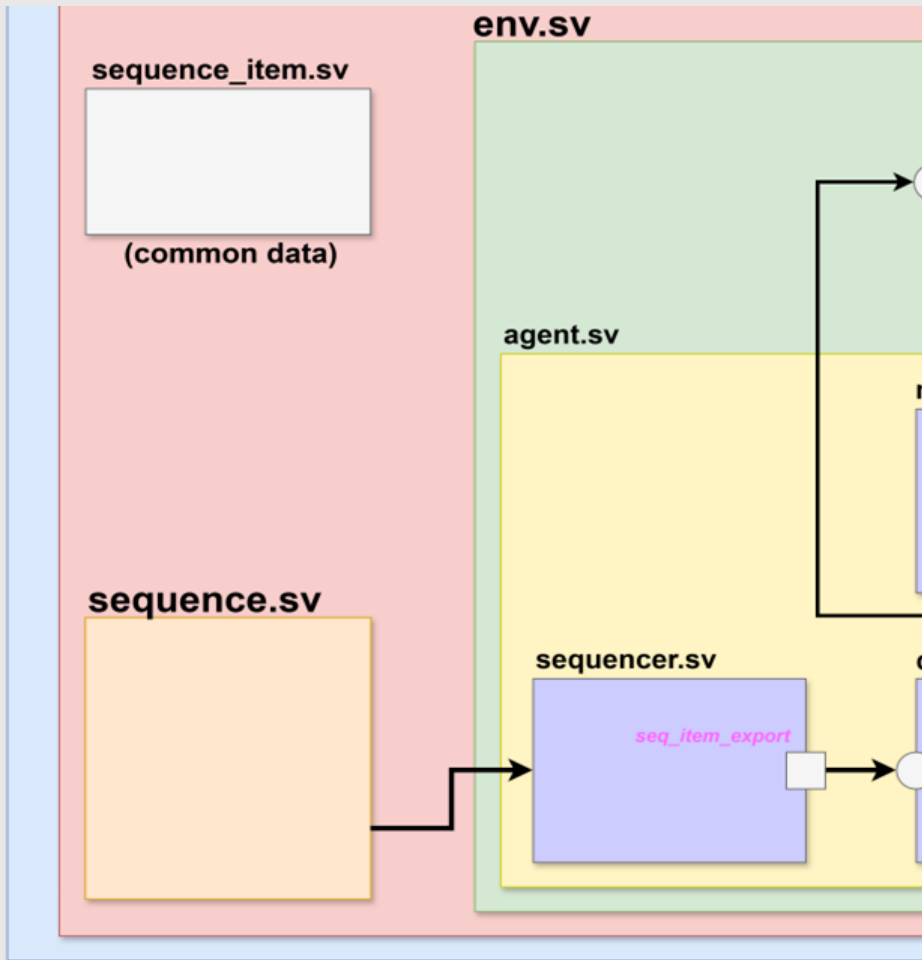
- Sequence Item : BUS로 한 번에 보내는 데이터 묶음
- Sequence : Sequence Item들을 어떤 순서로 보낼지 적어 둔 시나리오



- UVM_Sequence_base
 - 핵심동작 Engine으로 실행 주기, 시퀀스 언제 시작하고 종료되는지 관리 등의 Phase, 함수가 존재
 - Pre_Start, Start, Post_start, Pre_do, Mid do, Post_do, Start_item, Finish_item, lock, grab, set_priority, get_piority
- UVM_Sequence
 - Request/Response타입으로 핵심 기능은 Wrapping해서 사용자가 사용하게 쉽게 인터페이스만 추가함
 - Set_sequencer, Start, Get_response, Put_response, Get_current_item

2) Sequence Usedetail(1)

- Sequence: 특정 Stimulus를 DUT에 전달하기 위해 여러가지 방식으로 Sequence 사용가능
- Case1 : Base Sequence Use



```
// --- Base Sequence 정의 (start_item/finish_item 사용) ---
class base_sequence extends uvm_sequence #(my_data);
    uvm_object_utils (base_sequence)
    uvm_declare_p_sequencer (my_sequencer) // p_sequencer 변수 선언

    my_data data_obj;
    int unsigned n_times = 3; // 기본 횟수 1 -> 3으로 변경하여 실행 확인 용어

    function new (string name = "base_sequence");
        super.new (name);
    endfunction

    virtual task pre_body ();
        uvm_info ("BASE_SEQ", $sformatf ("Optional code can be placed here in pre_body()", UVM_MEDIUM)
        // phase가 null이 아닐 때만 objection을 raise/drop해야 안전합니다.
        if (starting_phase != null) begin
            starting_phase.raise_objection (this);
            uvm_info ("BASE_SEQ", "Objection Raised in pre_body()", UVM_MEDIUM)
        end
    endtask

    virtual task body ();
        uvm_info ("BASE_SEQ", $sformatf ("Starting body of %s, running %0d times", this.get_name(), n_times),
        UVM_MEDIUM)

        // data_obj는 반복문 외부에서 한 번만 생성합니다.
        data_obj = my_data::type_id::create ("data_obj");

        repeat (n_times) begin
            // 전송 요청 (Sequencer에 Item을 보낼 준비)
            start_item (data_obj);

            // 응답화
            assert (data_obj.randomize ())
            else 'uvm_error(get_name(), "Randomization failed");

            // 전송 완료 및 Driver의 item_done 대기
            finish_item (data_obj);
        end
        uvm_info (get_type_name (), $sformatf ("Sequence %s is over", this.get_name()), UVM_MEDIUM)
    endtask

    virtual task post_body ();
        uvm_info ("BASE_SEQ", $sformatf ("Optional code can be placed here in post_body()", UVM_MEDIUM)
        if (starting_phase != null) begin
            starting_phase.drop_objection (this);
            uvm_info ("BASE_SEQ", "Objection Dropped in post_body()", UVM_MEDIUM)
        end
    endtask
endclass
```

p_sequencer를 사용하는
주된 이유는 sequence는 실행될 때 어떤 sequencer에 연결될지 모르기 때문

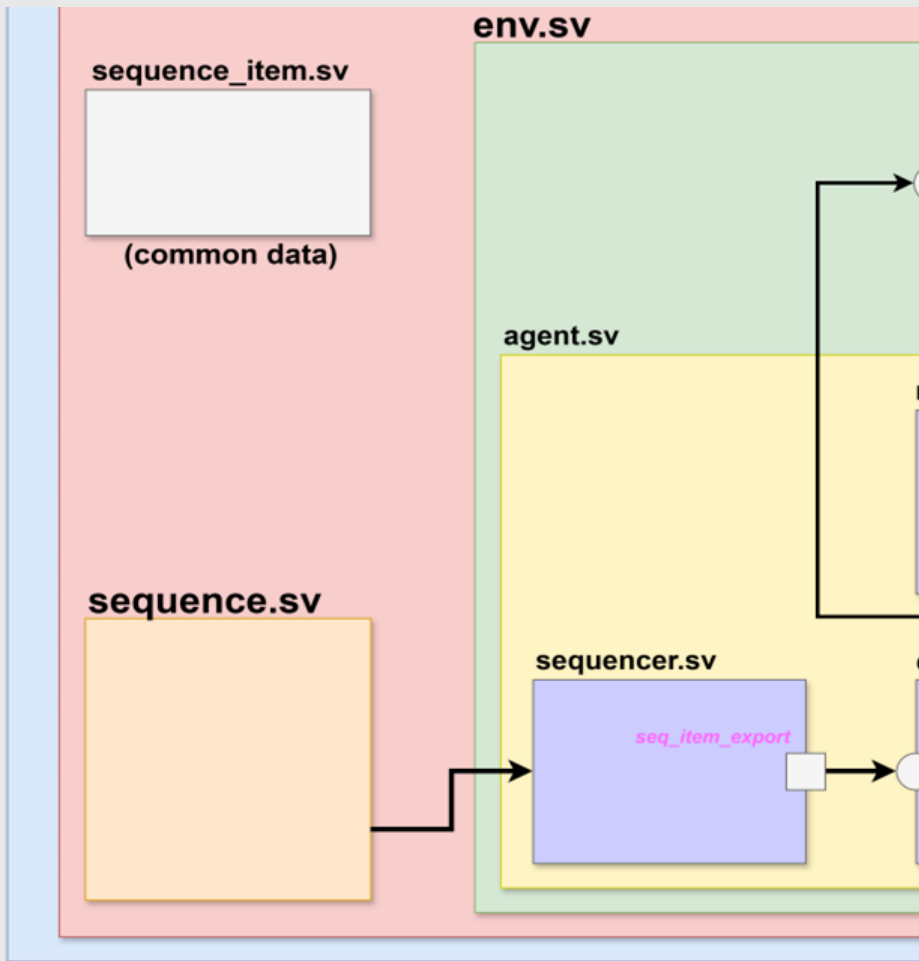
3번이면 sequence, sequencer, driver간 통신이 반복적, 정상적 확인

Body Task 실행 전
Raise_objection을 호출하여
테스트 종료 방지 및 Body에
DUT 자극을 보내기 전 필요한
환경 조건 충족 확인

Data object 생성 후
Handshake를 통해 DATA 전송

3) Sequence Usedetail(2)

- Sequence: 특정 Stimulus를 DUT에 전달하기 위해 여러가지 방식으로 Sequence 사용가능
- Case1 : UVM Sequence Use



```
class write_sequence extends uvm_sequence #(fsm_rw_data);
    uvm_object_utils(write_sequence)
    function new(string name = "write_sequence"); super.new(name); endfunction
    task body();
        fsm_rw_data pkt;
        uvm_do_with(pkt, { addr == 8'H0; });
        uvm_info(get_name(), $sformatf("WRITE executed: ADDR=0x%h, DATA=0x%h", pkt.addr, pkt.data), UVM_LOW);
    endtask
endclass

class fsm_change_sequence extends uvm_sequence #(fsm_rw_data);
    uvm_object_utils(fsm_change_sequence)
    function new(string name = "fsm_change_sequence"); super.new(name); endfunction
    virtual task body();
        fsm_rw_data pkt;
        uvm_do_with(pkt, { addr == 8'H0; data == 8'HCC; });
    endtask
endclass
```

Write & Sequence(FSM 제어 영역인 0x0 ~ 0x3을 피하고 데이터 영역에 Write)

```
// --- Sequence Library 모듈: Virtual Sequence를 포함한 가중치 기반 선택 모듈 ---
class random_fsm_runner extends uvm_sequence #(fsm_rw_data);
    uvm_object_utils(random_fsm_runner)

    // Sequence가 실행될 총 횟수
    rand int num_sequences = 3;

    function new(string name = "random_fsm_runner"); super.new(name); endfunction

    virtual task body();
        write_sequence m_write_seq;
        fsm_change_sequence m_fsm_seq;

        for (int i = 0; i < num_sequences; i++) begin
            // 가중치 기반 랜덤 선택 로직 (Write 3, FSM Change 1)
            // SystemVerilog의 randcase 블록은 가중치(weight)에 따라 선택을 수행
            case (1)
                // 가중치 3: Write Sequence 실행
                3: begin
                    m_write_seq = write_sequence::type_id::create("m_write_seq");
                    m_write_seq.start(m_sequencer, this);
                    uvm_info(get_name(), $sformatf("Sequence #0d: Selected WRITE (weight 3)", i), UVM_LOW);
                end
                // 가중치 1: FSM Change Sequence 실행
                1: begin
                    m_fsm_seq = fsm_change_sequence::type_id::create("m_fsm_seq");
                    m_fsm_seq.start(m_sequencer, this);
                    uvm_info(get_name(), $sformatf("Sequence #0d: Selected FSM CHANGE (weight 1)", i), UVM_LOW);
                end
            default: begin end
        endcase
    endtask
endclass
```

Num_sequence 횟수만큼 Run & Weight는 Randcase에 따라 확률적으로 선택



8.

UVM RAL

08

1) Register Abstraction Layer

● What is RAL?

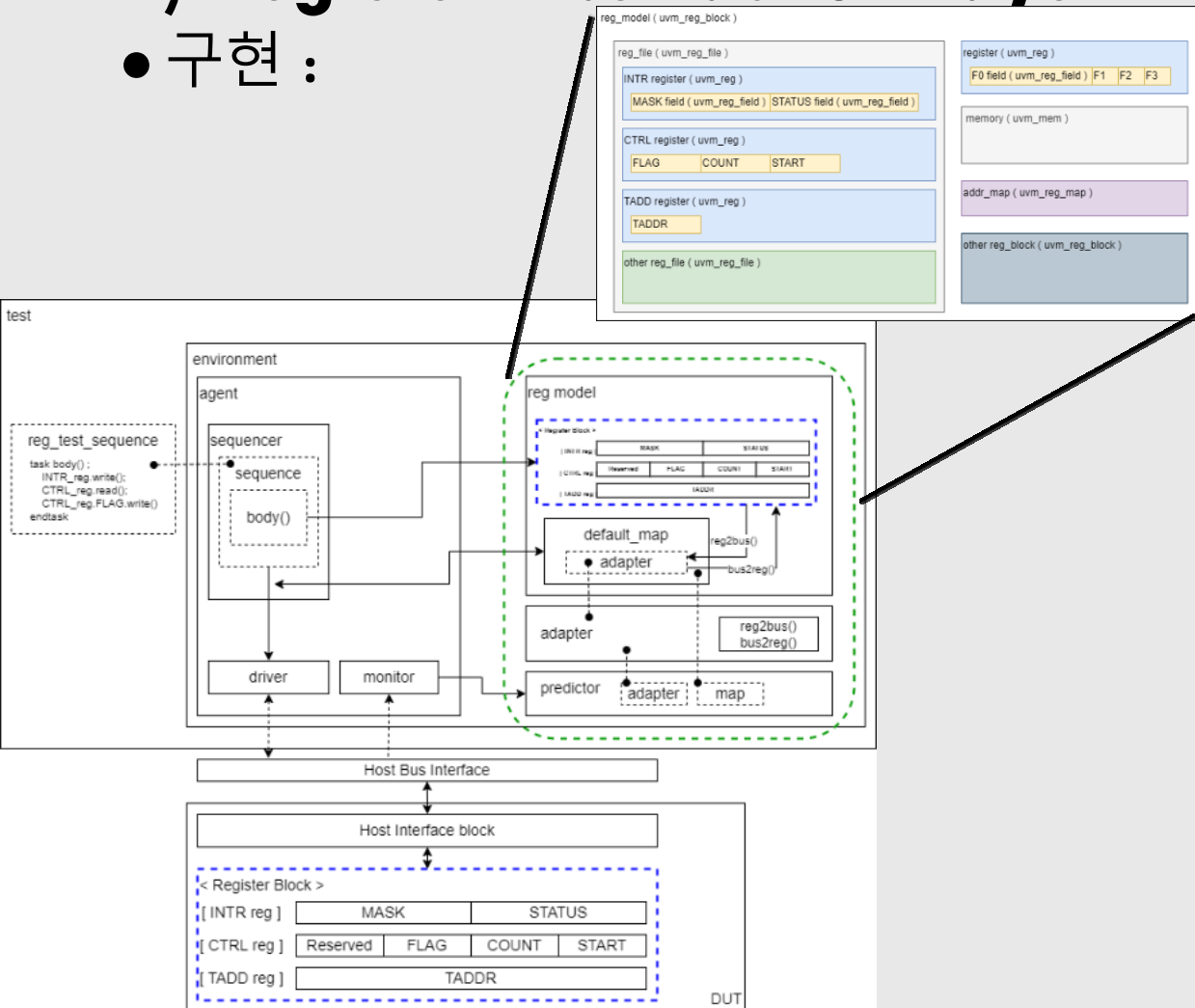
- Verification 하는 DUT의 register/Memory map을 모델링 하기 위해 제공된 base class들과 관련 API
- Digital 설계 내에서 Register와 memory mapping 구조를 modeling하고 검증하는 표준화 방법을 제공

● Why RAL?

- TB에 register, memory를 추상화해서 주소, Bit 위치를 직접 다루지 않고 객체 기반으로 읽기/쓰기/미러링을 가능하게 하는 계층
- Register Model Environment
 - Reg env -> TB의 env 내부에서 instance
 - 설계 레지스터 값을 반영하는 UVM Class 기반 Register Model
 - RAL Transaction과 Bus Transaction을 변환하는 Adapter
 - Bus Monitor에서 본 실제 Transaction으로 RAL Mirror를 Update하는 predictor

1) Register Abstraction Layer

구현 :



● Step1. Spec 분석

- DUT의 register read/write 동작 확인

● Step2. Register Model 작성

- register, address map에 따라서 register model을 UVM에서 제공하는 class(register_block, register file, register, register field, memory 등)를 활용하여 작성

● Step3. Adapter

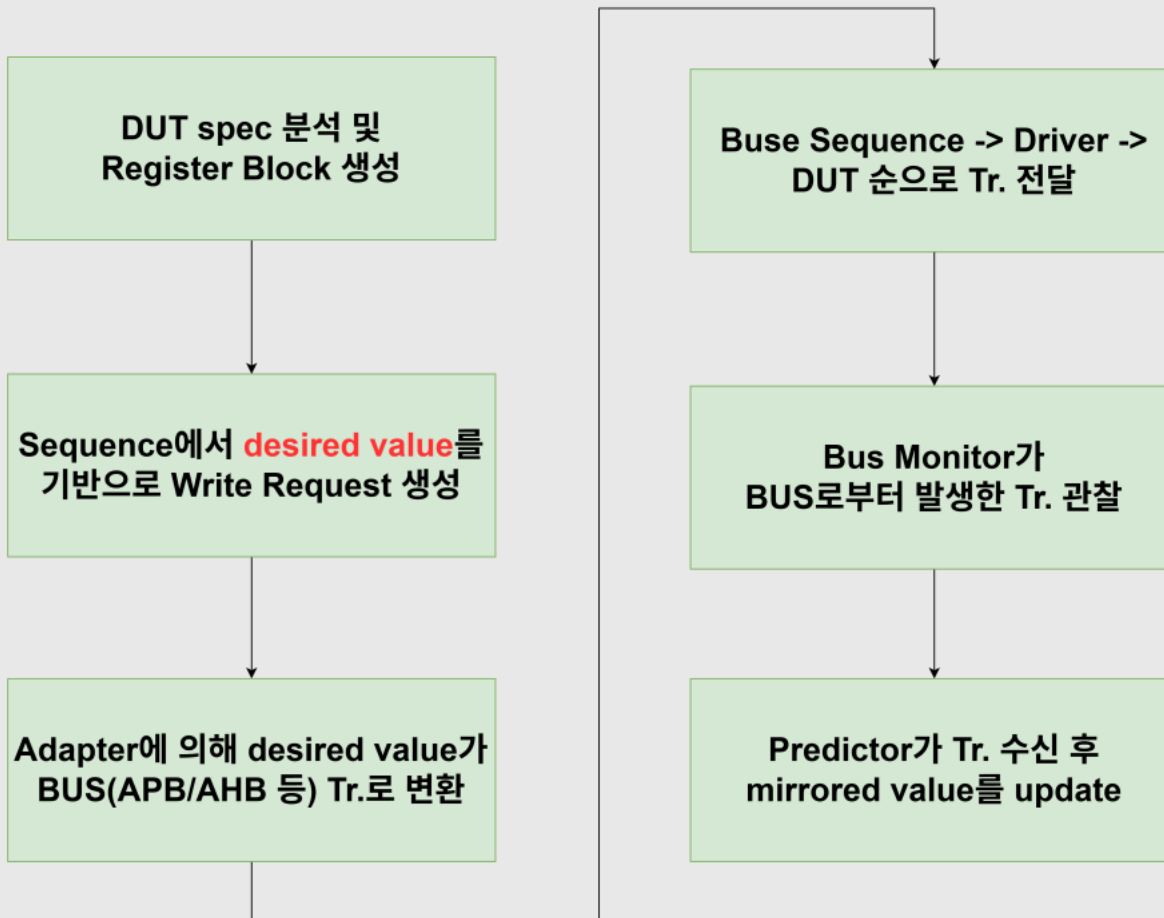
- Register model 및 bus agent의 sequencer를 연결하기 위한 adapter 작성
- Sequence에서 주는 write(), read() 명령문을 BUS Transaction으로 변환하는 역할을 담당

● Step4. Predictor

- TB의 register block에서 register를 바꾸면, DUT에서는 이를 어떻게 감지할까? -> predictor를 통해 sync가 이루어짐!

1) Register Abstraction Layer

- RAL 동작 흐름 :



1.Desired Reg vs Mirrored Reg

a. Desired Reg

- i. user에 의해 access 되는 부분으로 DUT register들에 바라는 값들(write value)

b.Mirrored Reg

- i. 항상 DUT로부터의 실제 값을 반영.

1) Register Abstraction Layer

● 실제 코드 기반 구현.

1 : DUT Spec 분석 & Register Block 생성

```
class reg_ctl extends uvm_reg;
  rand uvm_reg_field En;
  rand uvm_reg_field Mode;
  rand uvm_reg_field Halt;
  rand uvm_reg_field Auto;
  rand uvm_reg_field Speed;
  rand uvm_reg_field Reserved_6_10; // LSB 6~10 (5비트) 커버
  rand uvm_reg_field Reserved_16_31; // LSB 16~31 (16비트) 커버

  `uvm_object_utils(reg_ctl)
  function new (string name = "reg_ctl");
    super.new (name, 32, UVM_NO_COVERAGE);
  endfunction
  virtual function void build ();
    this.En = uvm_reg_field::type_id::create ("En");
    this.Mode = uvm_reg_field::type_id::create ("Mode");
    this.Halt = uvm_reg_field::type_id::create ("Halt");
    this.Auto = uvm_reg_field::type_id::create ("Auto");
    this.Speed = uvm_reg_field::type_id::create ("Speed");
    this.Reserved_6_10 = uvm_reg_field::type_id::create ("Reserved_6_10");
    this.Reserved_16_31 = uvm_reg_field::type_id::create ("Reserved_16_31");

    // 기존 필드 구성
    this.En.configure (this, 1, 0, "RW", 0, 1'h0, 1, 1, 1);
    this.Mode.configure (this, 3, 1, "RW", 0, 3'h2, 1, 1, 1);
    this.Halt.configure (this, 1, 4, "RW", 0, 1'h1, 1, 1, 1);
    this.Auto.configure (this, 1, 5, "RW", 0, 1'h0, 1, 1, 1);

    // LSB 6부터 10까지 5비트를 명시적으로 Reserved 처리
    this.Reserved_6_10.configure (this, 5, 6, "RW", 0, 5'h0, 1, 1, 1);

    this.Speed.configure (this, 5, 11, "RW", 0, 5'h1c, 1, 1, 1);

    // LSB 16부터 31까지 16비트를 명시적으로 Reserved 처리
    this.Reserved_16_31.configure (this, 16, 16, "RW", 0, 16'h0, 1, 1, 1);
```

2 : Sequence에서 desired value 기반 write 요청

```
`uvm_info("TEST", $sformatf("Writing 0x%h to CTRL (Addr 0x0)", write_data),
UVM_MEDIUM)
m_ral_model.m_reg_ctl.write(status, write_data, UVM_FRONTDOOR);
```

3 : Adapter에 의해 BUS Tr로 변환

```
class reg2apb_adapter extends uvm_reg_adapter;
  `uvm_object_utils (reg2apb_adapter)
  function new (string name = "reg2apb_adapter");
    super.new (name);
    supports_byte_enable = 0;
    provides_responses = 0;
  endfunction

  virtual function uvm_sequence_item reg2bus (const ref uvm_reg_bus_op rw);
    bus_pkt pkt = bus_pkt::type_id::create ("pkt");
    pkt.write = (rw.kind == UVM_WRITE) ? 1: 0;
    pkt.addr = rw.addr;
    pkt.data = rw.data;
    return pkt;
  endfunction
```

1) Register Abstraction Layer

- 실제 코드 기반 구현.

4. BUS sequence -> Driver -> DUT로 tr 전달

```
class my_driver extends uvm_driver #(bus_pkt);  
  `uvm_component_utils(my_driver)  
  function new(string name, uvm_component parent); super.new(name, parent);  
endfunction  
  virtual task run_phase(uvm_phase phase);  
    forever begin  
      bus_pkt req;  
      seq_item_port.get_next_item(req);  
      `uvm_info(get_full_name(),  
        $sformatf("Driving %s Tr: Addr=0x%0h, Data=0x%0h", req.write ?  
"WRITE" : "READ", req.addr, req.data), UVM_HIGH)  
      #10ns;  
      seq_item_port.item_done();  
    end  
  endtask  
endclass
```

5 : Bus Monitor가 실제 DUT로부터의 Tr 관찰

```
virtual task run_phase(uvm_phase phase);  
  forever begin  
    bus_pkt pkt = bus_pkt::type_id::create ("pkt");  
    pkt.addr = `UVM_REG_ADDR_WIDTH'h0;  
    pkt.data = 32'hCAFE_BEEF;  
    pkt.write = 0;  
    #20ns;
```

6 : Predictor가 monitor로부터 tr를 수신 후, mirrored value를 register model에 update

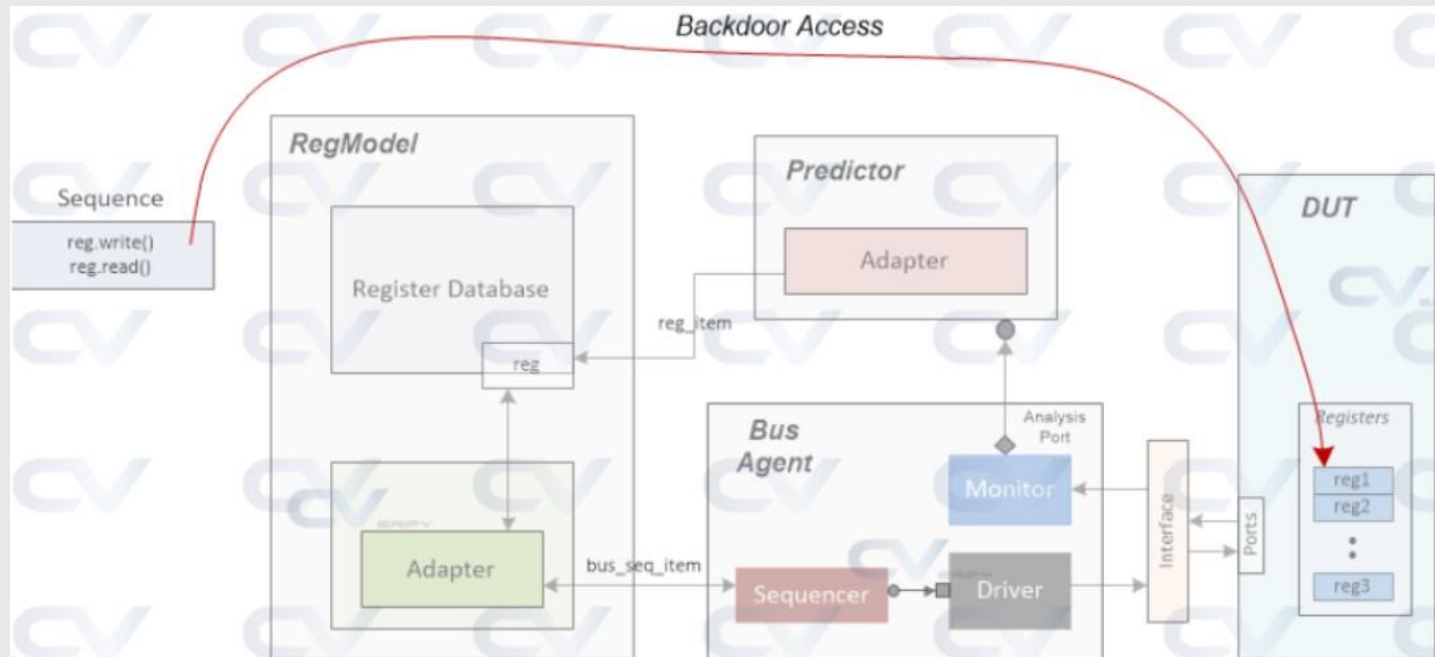
```
// 예측기 연결: 모니터 -> 예측기  
m_agent.m_mon.ap.connect(m_apb_predictor.bus_in);
```

```
// 예측기에 RAL 맵과 어댑터 연결  
m_apb_predictor.map = m_ral_model.default_map;  
m_apb_predictor.adapter = m_apb_adapter;
```

1) Register Abstraction Layer(BackDoor)

- 구현

- DUT의 Simulator의 Database를 사용하여 DUT 내부의 신호에 직접 접근
- UVM의 Register Model의 DUT의 실제 HDL 변수와 직접 동기화
- 그렇다면, 가져온 TB의 RegBlock에 특정 값을 Write/Read하는게 실제 DUT의 RegBlock에 쓰는 것과 어떻게 연동이 될까?
 - > RAL의 HDL(직접 경로)를 통해 접근



1) Register Abstraction Layer(BackDoor)

• 동작 흐름.

1 : DUT Register 사양 추상화 및 Desired Value 저장 및 Model 생성

```
class reg_block extends uvm_reg_block;
  rand reg_ctl    m_reg_ctl;
  rand reg_stat   m_reg_stat;
  rand reg_inten  m_reg_inten;

  `uvm_object_utils(reg_block)

  function new (string name = "reg_block");
    super.new (name, UVM_NO_COVERAGE);
    // 블록 레벨의 백도어 경로 루트만 설정
    this.set_hdl_path_root("tb_ral_model.dut");
  endfunction

  virtual function void build ();
    this.default_map = create_map ("", 0, 4, UVM_LITTLE_ENDIAN, 0);

    this.m_reg_ctl = reg_ctl::type_id::create ("m_reg_ctl");
    this.m_reg_stat = reg_stat::type_id::create ("m_reg_stat");
    this.m_reg_inten = reg_inten::type_id::create ("m_reg_inten");

    this.m_reg_ctl.configure (this, null);
    this.m_reg_stat.configure (this, null);
    this.m_reg_inten.configure (this, null);

    this.m_reg_ctl.build();
    this.m_reg_stat.build();
    this.m_reg_inten.build();
    // 레지스터 맵의 add_path/add_hdl_path 호출을 제거하여 컴파일 오류 해결
    // 레지스터들은 부모 블록(reg_block)의 root 경로를 상속받으며,
    // 레지스터 인스턴스 이름이 자동으로 경로의 끝(tb_ral_model.dut.m_reg_ctl)으로 사용
    // 되도록 가정
    this.default_map.add_reg (this.m_reg_ctl, `UVM_REG_ADDR_WIDTH'h0, "RW",
0);
    this.default_map.add_reg (this.m_reg_stat, `UVM_REG_ADDR_WIDTH'h4, "RO",
0);
    this.default_map.add_reg (this.m_reg_inten, `UVM_REG_ADDR_WIDTH'h8, "RW",
0);
  endfunction
endclass
```

2 : Register의 Run Phase에서 Desired Value 설정 및 Transaction Start

```
`uvm_info("TEST", $sformatf("Writing 0x%h via Backdoor to CTRL (Addr
0x0)", expected_data), UVM_MEDIUM)
// UVM_DEFAULT_PATH를 사용하여 백도어 경로를 통해 레지스터에 Write
// 경로: tb_ral_model.dut.m_reg_ctl (레지스터 인스턴스 이름이 경로의 일부로 사용
됨)
m_ral_model.m_reg_ctl.write(status, expected_data, UVM_BACKDOOR);
```

3 : reg2bus를 이용하여 RAL 작업정보를 tr로 변환

```
virtual function uvm_sequence_item reg2bus (const ref uvm_reg_bus_op rw);
  bus_pkt pkt = bus_pkt::type_id::create ("pkt");
  pkt.write = (rw.kind == UVM_WRITE) ? 1: 0;
  pkt.addr = rw.addr;
  pkt.data = rw.data;
  return pkt;
endfunction
```

1) Register Abstraction Layer(BackDoor)

- 동작 흐름.

4. Transaction을 받아 Driver로 전달

```
virtual task run_phase(uvm_phase phase);
    forever begin
        bus_pkt req;
        seq_item_port.get_next_item(req);
        `uvm_info(get_full_name(),
            $sformatf("Driving %s Tr: Addr=0x%0h, Data=0x%0h", req.write ?
"WRITE" : "READ", req.addr, req.data), UVM_HIGH)
        #10ns;
        seq_item_port.item_done();
    end
endtask
```

5 : Bus Interface를 Monitoring하여 DUT로
전송되는 Transaction Capture

```
class my_monitor extends uvm_monitor;
    `uvm_component_utils (my_monitor)
    uvm_analysis_port #(bus_pkt) ap;
    function new (string name="my_monitor", uvm_component parent);
        super.new (name, parent);
        ap = new ("ap", this);
    endfunction
    virtual task run_phase(uvm_phase phase);
        // 백도어 액세스 시 모니터 로직은 사용되지 않습니다.
    endtask
endclass
```

6 : Monitor가 Capture한 Transaction을 수신
하여 다시 RAL 형식으로 변환

```
virtual function void bus2reg (uvm_sequence_item bus_item, ref uvm_reg_bus_op
rw);
    bus_pkt pkt;
    if (! $cast (pkt, bus_item)) begin
        `uvm_fatal ("reg2apb_adapter", "Failed to cast bus_item to pkt")
    end
    rw.kind = pkt.write ? UVM_WRITE : UVM_READ;
    rw.addr = pkt.addr;
    rw.data = pkt.data;
    rw.status = UVM_IS_OK;
    `uvm_info("ADAPTER", $sformatf("bus2reg: Predictor sees Addr=0x%h,
Data=0x%h, Kind=%s", rw.addr, rw.data, rw.kind.name()), UVM_HIGH)
endfunction
```

```
// 예측기 연결: 모니터 -> 예측기
m_agent.m_mon.ap.connect(m_apb_predictor.bus_in);
```

```
// 예측기에 RAL 맵과 어댑터 연결
m_apb_predictor.map = m_ral_model.default_map;
m_apb_predictor.adapter = m_apb_adapter;
```

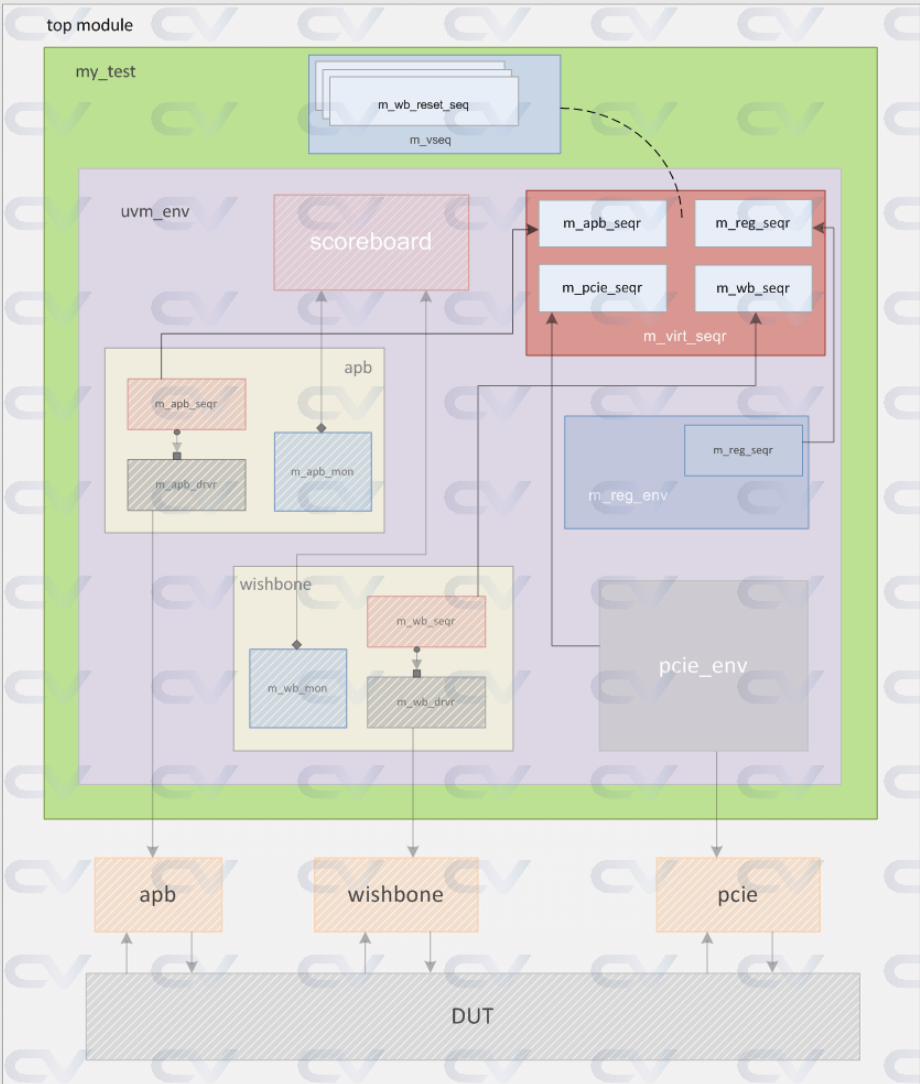


9.

Virtual Sequencer, Virtual Sequence

09

1) Virtual Sequence(top sequence)



What is Virtual Sequence?

- 여러 개의 sequence(= 여러 agent)를 동시에/순차적으로 제어하기 위한 상위 sequence
- transaction을 직접 driver로 보내지 않고, 다른 sequence들을 지휘하는 sequence.
- Test 모듈에서 생성 및 실행

Features

- 다중의 agent들의 sequence 실행을 control (transaction을 직접 driving하지는 않음)
- sequence들의 실행 순서를 미세하게 조정 가능
- agent와 관련이 없어서 sequence item type이 필요 없음.



Virtual Sequence & Virtual Sequencer

1) Virtual Sequence(top sequence)

Why Virtual Sequence?

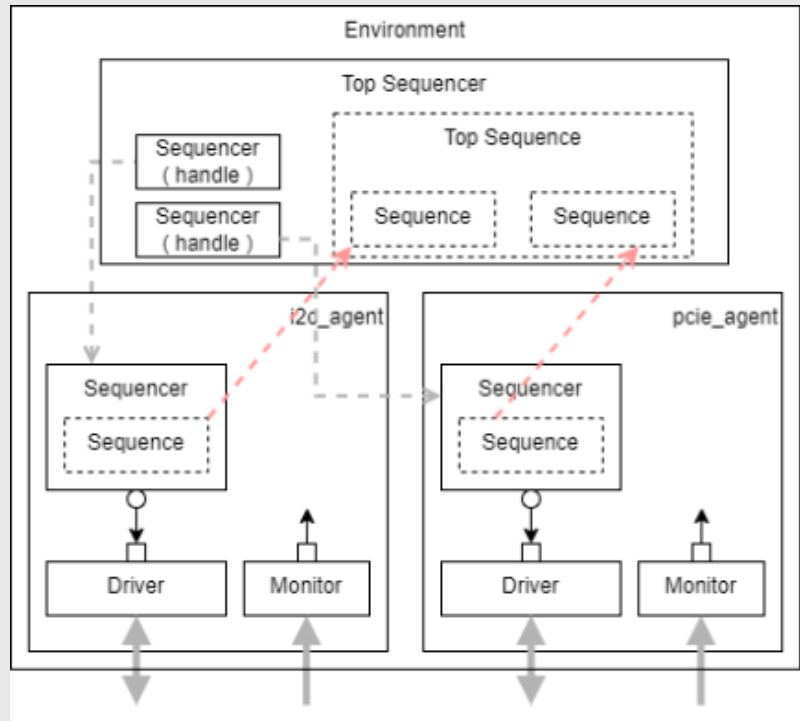
- sequence는 항상 하나의 sequencer에만 붙는다.



```
class i2c_seq extends uvm_sequence #(uart_item);
```

-> 이 sequence는 i2c sequencer 하나만 제어가 가능

-> SPI, APB, GPIO sequencer를 동시에 건드리는 건 불가능



• SoC Level :

- SoC에서는 i2c <-> GPIO, GPIO <-> UART 등 다양한 IP들끼리 통신
- 이러한 시스템 레벨에서의 동작에 대한 시나리오를 작성하려면 단일 sequence로는 표현 불가
- 여러 agent간에 넘나들 수 있어야함. **virtual sequence가 등장한 이유!**



Virtual Sequence & Virtual Sequencer

1) Virtual Sequence(top sequence)

How to Use Virtual Sequence?

```
class my_virtual_seq_extends uvm_sequence;
  `uvm_object_utils (my_virtual_seq)
  `uvm_declare_p_sequencer (my_virtual_sequencer)

  function new (string name = "my_virtual_seq");
    super.new (name);
  endfunction

  apb_rd_wr_seq  m_apb_rd_wr_seq;
  wb_reset_seq  m_wb_reset_seq;
  pcie_gen_seq  m_pcie_gen_seq;

  task pre_body();
    m_apb_rd_wr_seq = apb_rd_wr_seq::type_id::create ("m_apb_rd_wr_seq");
    m_wb_reset_seq  = wb_reset_seq::type_id::create ("m_wb_reset_seq");
    m_pcie_gen_seq  = pcie_gen_seq::type_id::create ("m_pcie_gen_seq");
  endtask

  task body();
    ...
    m_apb_rd_wr_seq.start (p_sequencer.m_apb_seqr);
    fork
      m_wb_reset_seq.start (p_sequencer.m_wb_seqr);
      m_pcie_gen_seq.start (p_sequencer.m_pcie_seqr);
    join
    ...
  endtask
endclass
```

- my_virtual_seq는 uvm_sequence를 상속
- p_sequencer라고 불리는 handle을 'uvm_declare_p_sequencer macro를 활용해 생성
- virtual_sequence에서 virtual_sequencer에 접근하겠다는 선언

- Virtual sequence에 포함되는 각각의 sequence들은, start() method를 통해 각각에 대응되는 sequencer와 연결.
- M_sequencer
 - 특정 sequence를 실제로 start()시킨 sequencer에 대한 핸들
 - 직접 선언하지 않아도, uvm_sequence안에 자동으로 존재

• test

```
class my_test_extends uvm_test;
  `uvm_component_utils (my_test)

  my_env  m_env;
  ...

  task run_phase (uvm_phase phase);
    my_virtual_seq m_vseq = my_virtual_seq::type_id::create ("m_vseq");
    phase.raise_objection (this);
    m_vseq.start (m_env.m_virtual_seqr);
    phase.drop_objection (this);
  endtask
endclass
```

- Virtual sequence가 정의되면, test 모듈의 run_phase에서 시작가능.



Virtual Sequence & Virtual Sequencer

2) Virtual Sequencer(top sequencer)

p_sequencer

- virtual sequence가 “자신이 어떤 virtual sequencer와 연결되어 동작하는지”를 알 수 있도록 만들어주는 핸들.
- ``uvm_declare_p_sequencer`는 p_sequence를 자동으로 선언 + 캐스팅 + 연결까지 해주는 메크로

```
class my_virtual_seq extends uvm_sequence;
    `uvm_object_utility(my_virtual_seq)
    `uvm_declare_p_sequencer(my_virtual_sequencer)

    function new(string name = "my_virtual_seq");
        super.new(name);
    endfunction

    apb_rd_wr_seq    m_apb_rd_wr_seq;
    wb_reset_seq     m_wb_reset_seq;
    pcie_gen_seq     m_pcie_gen_seq;

    task pre_body();
        m_apb_rd_wr_seq = apb_rd_wr_seq::type_id::create("m_apb_rd_wr_seq");
        m_wb_reset_seq   = wb_reset_seq::type_id::create("m_wb_reset_seq");
        m_pcie_gen_seq   = pcie_gen_seq::type_id::create("m_pcie_gen_seq");
    endtask

    task body();
        ...
        m_apb_rd_wr_seq.start(p_sequencer.m_apb_seqr);
        fork
            m_wb_reset_seq.start(p_sequencer.m_wb_seqr);
            m_pcie_gen_seq.start(p_sequencer.m_pcie_seqr);
        join
        ...
    endtask
endclass
```

- P_sequencer는 특정 sequence를 실행시킨 sequencer 객체에 대한 handle.
- 즉, 특정 virtual sequence가 어떤 sequencer에서 start() 되었는지, 그 start된 sequencer를 정확한 타입으로 접근하기 위한 포인터.
- ``uvm_declare_p_sequencer`
 - P_sequencer 선언
 - Start() 시점에 실행시킨 sequencer를 my_virtual_sequencer 타입으로 캐스팅
 - 캐스팅 실패시 에러처리



Virtual Sequence & Virtual Sequencer

2) Virtual Sequencer(top sequencer)

실제 구조 - 여러 sequence를 포함하는 virtual sequencer 선언

```
class my_virtual_seq extends uvm_sequence;
    `uvm_object_utils (my_virtual_seq)
    `uvm_declare_p_sequencer (my_virtual_sequencer)

    function new (string name = "my_virtual_seq");
        super.new (name);
    endfunction

    apb_rd_wr_seq    m_apb_rd_wr_seq;
    wb_reset_seq     m_wb_reset_seq;
    pcie_gen_seq     m_pcie_gen_seq;

    task pre_body();
        m_apb_rd_wr_seq = apb_rd_wr_seq::type_id::create ("m_apb_rd_wr_seq");
        m_wb_reset_seq   = wb_reset_seq::type_id::create ("m_wb_reset_seq");
        m_pcie_gen_seq   = pcie_gen_seq::type_id::create ("m_pcie_gen_seq");
    endtask

    task body();
        ...
        m_apb_rd_wr_seq.start (p_sequencer.m_apb_seqr);
        fork
            m_wb_reset_seq.start (p_sequencer.m_wb_seqr);
            m_pcie_gen_seq.start (p_sequencer.m_pcie_seqr);
        join
        ...
    endtask
endclass
```

1. uvm_sequence를 상속받아 sequence를 정의
2. p_sequencer라고 불리는 handle을 ``uvm_declare_p_sequencer` macro를 활용해 생성
3. virtual sequence에 포함되는 각각의 sequence는 각각에 상응하는 sequencer와 연동되어 동작하며, start() 메소드를 활용해 시작.
4. 각각의 sequencer는 p_sequencer(virtual sequencer를 가르키는 p_sequencer handle)을 참조